

Discovering Antagonistic Near-Balanced Dense Subgraphs in Signed Networks

Xiaojia Xu^{ID}, Haoyu Liu, Xiaowei Lv^{ID}, Yongcai Wang^{ID}, and Deying Li^{ID}

Abstract—Detecting antagonistic near-balanced dense subgraphs is a crucial problem for community search and conflict detection in graph and network analysis, which has wide applications in social media analysis, business, geopolitics etc. To tackle the absence of a unified definition of antagonistic situation, we propose the *antagonism* measure, which quantifies the quality of subgraphs by three dimensions: polarity, internal cohesion, and external antagonistic normalized density. Inspired by the contribution of small antagonistic balanced patterns to antagonism and balance, this paper introduces an efficient algorithmic framework to mine locally specific pattern densest subgraph structure to find subgraphs with high antagonism. We in particular jointly consider h -pattern compact number and L h -PDS and design a new Iterative Propose-Prune-and-Verify pipeline in signed graphs (IPPV-S) for top- k L h -PDS detection. The key contributions are: (1) The antagonism measure is defined, bridging the gap between structural density and balance theory. (2) An efficient algorithmic pipeline that combines convex optimization with maximum flow verification is proposed, which enables scalable and efficient antagonistic near-balanced dense subgraph discovery. (3) Extensive experiments on real signed network datasets show the effectiveness of our approach in uncovering meaningful subgraphs that capture both cooperative and conflicting dynamics.

Index Terms—Data mining, dense subgraph, signed network.

I. INTRODUCTION

SIGNED networks, where edges are annotated with positive or negative signs to represent friendly or antagonistic relationships, have become a powerful tool for modeling complex systems in various domains, such as social interactions [1] [2], biological processes [3], and recommendation systems [4]. One fundamental theoretical framework for understanding signed networks is the balance theory [5] [6], which posits that the friend of a friend is a friend and the friend of an enemy is an

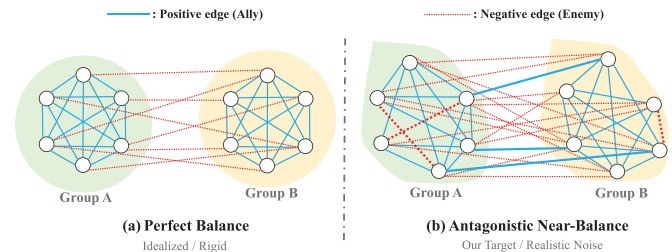


Fig. 1. Illustrative comparison between (a) Perfect Balance and (b) Antagonistic Near-Balance (Thicker lines in (b) denote violations of perfect balance).

enemy. Theoretically, a signed graph satisfies perfect balance [7] [8] if it can be perfectly partitioned into two opposing groups where all internal edges are positive and all crossing edges are negative. As illustrated in Fig. 1(a), this represents an idealized state where two factions are strictly separated with no internal disagreements or external alliances. However, real-world networks rarely exhibit such rigid patterns due to noise and complex interactions. For instance, consider an online political debate network. A state of perfect balance would imply two rigid parties with zero internal dissent, which is unrealistic. In contrast, real-world communities often exhibit antagonistic near-balanced structures, as shown in Fig. 1(b). Here, members of a political faction maintain strong internal cohesion and shared hostility toward an opposing faction, despite occasional internal disputes (highlighted by the thick red dashed line) or strategic external cooperation (highlighted by the thick blue solid line).

The potential applications are as follows. **Business and Geopolitics:** Both commercial relations between business organizations [9] and international relations between countries [10] can be modeled as signed graphs. The detection of antagonistic near-balanced dense subgraph helps to uncover strategically significant adversarial blocs. **Social Media Analysis:** User interactions on platforms like Facebook and Douban, modeled as signed graphs [2], [11], reveal polarized groups engaged in sustained disputes (e.g., ideologically opposed communities). Antagonistic near-balanced dense subgraph detection enables early identification of conflicts and prevents potential conflicts in real networks. **Natural Language Processing:** Word networks with synonyms and antonyms can be modeled as signed networks [12]. For example, the sets {good, great, excellent, nice, superior, superb} and {bad, worse, unfavorable, unrespectable, worst} represent clusters of synonymous words. The two sets can form a signed network, where dense intra-cluster positive

Received 31 July 2025; revised 6 February 2026; accepted 2 June 2026. Date of publication 15 June 2026; date of current version 8 August 2026. The work of Deying Li was supported in part by the National Natural Science Foundation of China under Grant 12071478. The work of Yongcai Wang was supported in part by the National Natural Science Foundation of China under Grant 61972404, in part by the Public Computing Cloud, Renmin University of China, and in part by the Blockchain Lab. School of Information, Renmin University of China. Recommended for acceptance by R. Akbarinia. (Corresponding author: Deying Li.)

Xiaojia Xu is with the Department of Software Engineering, Beijing University of Technology, Beijing 100124, China, and also with the School of Information, Renmin University of China, Beijing 100872, China (e-mail: xuxiaojia@ruc.edu.cn).

Haoyu Liu, Xiaowei Lv, Yongcai Wang, and Deying Li are with the School of Information, Renmin University of China, Beijing 100872, China (e-mail: hylu2187@ruc.edu.cn; lvxiaowei@ruc.edu.cn; ycw@ruc.edu.cn; dey-lingli@ruc.edu.cn).

Digital Object Identifier 10.1109/TKDE.2026.3703775

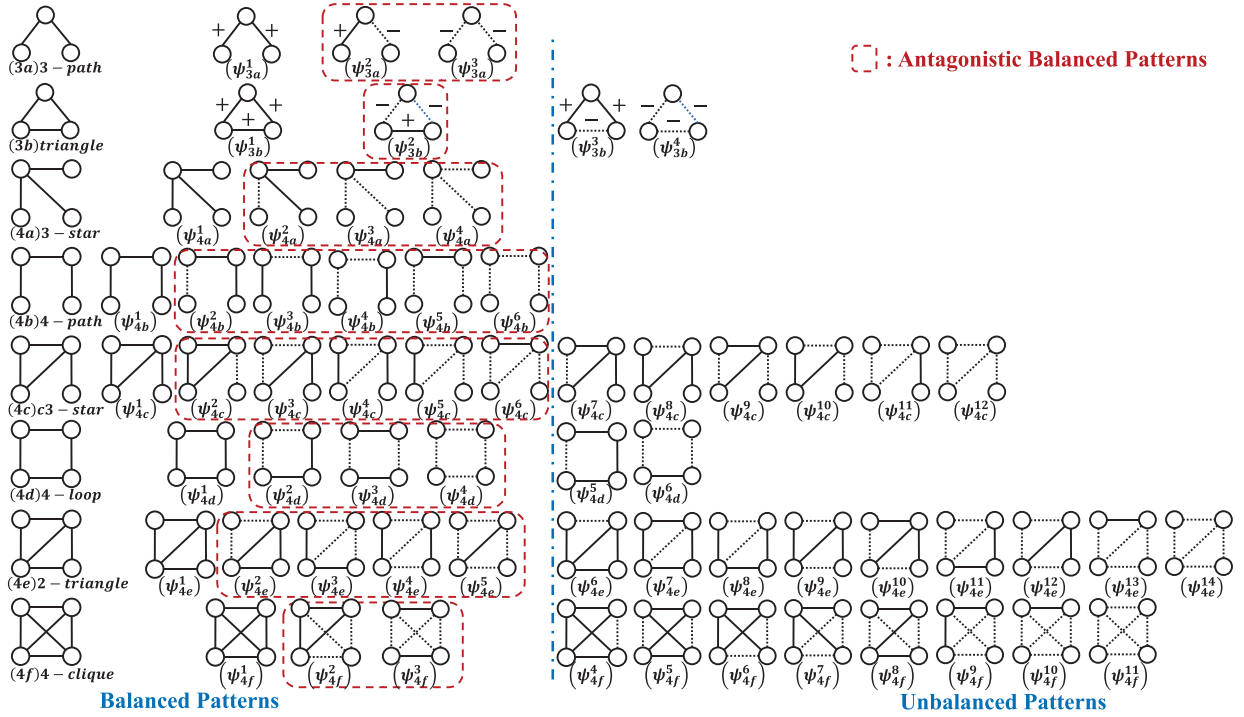


Fig. 2. An example of the signed subgraphs of all patterns with 3 and 4 vertices in G (ψ_{hx}^i denotes the i -th signed isomorphism type of the unsigned ψ_{hx}).

TABLE I
MAIN NOTATIONS

Notation	Definition
$G = (V, E)$	a graph with vertex set V and edge set E
n, m	$n = V , m = E $
$G[S]$	the subgraph induced by S
$\Psi_{hx}(G)$	the collection of all hx -patterns of G
$\psi_{hx}(V_{\psi_{hx}}, E_{\psi_{hx}})$	a particular kind of pattern x with h vertices
$d_{\psi_{hx}}(G)$	the hx -pattern density of G , $d_{\psi_{hx}}(G) = \frac{ \Psi_{hx}(G) }{ V }$
$\phi_{hx}(u)$	the hx -pattern compact number of vertex u
$\deg_G(v, \psi_{hx})$	the hx -pattern degree of vertex v in G
$\bar{\phi}_{hx}(u)$	an upper bound of $\phi_{hx}(u)$ in G
$\underline{\phi}_{hx}(u)$	a lower bound of $\phi_{hx}(u)$ in G

edges reflect synonymy, and inter-cluster negative edges capture antonymy. The detection of antagonistic near-balanced dense subgraph can be further applied to applications such as automatic question generation [13] and semantic extension [14].

The discovery of balanced subgraphs and antagonistic structures has attracted significant attention. However, prior works often either strictly adhere to balance theory [7], [8], [15], [16], failing to accommodate real-world noise, or focus on purely polarized structures [17], [18], [19] that may neglect topological density. Consequently, these approaches often overlook the subtle interplay between structural balance and density, failing to capture dense subgraphs that contain a mix of balanced and unbalanced relationships. To address this gap, we target antagonistic near-balanced dense subgraphs, which represent cohesive communities that are predominantly balanced but engaged in intense mutual conflict. To effectively quantify such structures, we propose the *antagonism* measure constructed from three

complementary dimensions. We select *polarity* to assess the adherence to structural balance, addressing the limitations of purely density-based methods. Simultaneously, we incorporate *internal cohesion* to enforce edge density, ensuring that the identified groups form tight communities rather than the sparse collections typically yielded by polarization-based methods. Finally, *external antagonistic normalized density* is introduced to explicitly measure the intensity of inter-group conflict. This dimension ensures that the detected subgraph is not merely an isolated balanced clique but is actively involved in strong adversarial interactions, as visually depicted by the dense red dashed lines in Fig. 1(b). Inspired by the contribution of small antagonistic balanced patterns to antagonism and balance, we introduce an efficient algorithmic framework to mine locally specific pattern densest subgraph structure to find the subgraphs with high antagonism. We formulate the problem as a convex optimization task centered around the pattern density. Specifically, we design an objective function that quantifies the tightness of the considered pattern within candidate subgraphs and employ a method to efficiently compute solutions. We use the *antagonism* measure to sort the output subgraphs, guaranteeing that the final output adheres to the desired properties of near-balance and density.

This paper makes three main contributions. **1)** We combine polarity, internal cohesion, and external antagonistic normalized density to define the *antagonism* measure, bridging the gap between structural density and balance theory. **2)** We propose an efficient algorithmic pipeline to find subgraphs with high antagonism for discovering antagonistic near-balanced dense subgraphs. The pipeline combines convex optimization with maximum flow verification, enabling efficient subgraph

discovery. **3)** Through extensive experiments on real-world signed networks, we demonstrate the effectiveness of our approach in uncovering meaningful subgraphs that capture both cooperative and conflicting dynamics.

Our findings not only advance the theoretical understanding of signed networks but also offer practical insights into applications such as community detection and conflict resolution. By providing a robust framework for analyzing the intricate balance of relationships in signed networks, this work lays the foundation for future research in this emerging area.

II. RELATED WORK

A. Antagonistic Dense Subgraph

The study on signed graphs was first proposed by Harary et al. [5], who introduced the concept of structural balance on signed graphs. The dense subgraph models on signed networks mainly include maximum balanced subgraph model, maximum balanced model and polarized community model.

Maximum balanced subgraph: Figueiredo et al. [7] first studied the maximum balanced subgraph problem and showed that the maximum balanced subgraph problem is an NP-hard problem. The pre-processing routines of the problem is discussed, and a greedy heuristic algorithm is proposed. Ordozgoiti et al. [15] proposed an efficient algorithm for finding large balanced subgraphs in signed networks, which relied on signed spectral theory and a novel bound for perturbations of the graph Laplacian, and the proposed algorithm is more scalable. However, maximum balanced subgraphs are often structurally sparse.

Maximum balanced clique: Researchers have defined different clique structures in signed networks, such as k -balanced trusted clique [20], (α, k) -clique [21], and maximum balanced clique [8], [16]. The concept of maximum balanced clique is proposed by Chen et al. [8], and they studied the maximal balanced clique enumeration problem and the maximum balanced clique search problem. The two problems are both NP-Hard. Yao et al. [16] also studied the two problems, and transformed the balanced clique problem on signed graphs to the maximum dichromatic clique problem on small subgraphs, improving the efficiency of the algorithm. However, the strict balance constraint often restricts these cliques to a trivial scale.

Antagonistic and polarized community: Zhang et al. [17] and Chu et al. [22] both studied the antagonistic communities with dense and positive edges in subgraphs and negative edges passing through different subgraphs. They proposed efficient algorithms to detect the corresponding structures. Yang et al. [23] proposed a new method for searching polarized communities on attribute signed networks based on random walk. Bonchi et al. [18] studied the problem of finding polarized communities in signed graphs, formulated the problem as a discrete eigenvector problem, and proved that the problem is NP-hard. Niu et al. [19] further improved the concept of polarized communities, defined a new metric for finding dense polarized communities, and proposed a heuristic algorithm based on balanced triangles. Recently, the field has further evolved to encompass signed graph representation learning [24] and degree-corrected

stochastic block models [25] for handling heterogeneous degree distributions. However, these methods do not satisfy the balance theory and cannot tell whether there is more agreement or more conflict.

B. Densest Subgraph and Its Variants

The DS problem is a classical problem in combinatorial optimization, which has various variants focusing on different aspects and different types of graphs.

Densest subgraph: The DS problem can be solved in polynomial time by exact methods based on maximum flow [26], [27], [28], linear programming [29], or convex optimization [30]. Exact algorithms cannot scale well to large graphs, so faster approximation algorithms for the DS problem are presented. Charikar [29] proposed a 2-approximation greedy peeling algorithm for the DS problem. Inspired by the multiplicative weights update method, Boob et al. [31] designed an iterative version of the greedy peeling algorithm, and the convergence of the algorithm is proved by Chekuri et al. [32].

h -clique densest subgraph: Tsourakakis [33] defined h -clique density and introduced the h -clique densest subgraph (CDS) problem. Mitzenmacher et al. [34] presented a sampling scheme called the densest subgraph sparsifier, yielding a randomized algorithm that produces a well-approximate solution to the CDS problem. Fang et al. [28] proposed more efficient exact algorithm and approximation algorithm for the CDS problem. Sun et al. [35] aimed at developing near-optimal and exact algorithms for the CDS problem on large real-world graphs. They modified the Frank-Wolfe algorithm for CDS and proved the effectiveness of the proposed algorithm.

Locally densest subgraph: The locally densest subgraph (LDS) problem is a variant of the densest subgraph (DS) problem. Qin et al. [36] proposed a method to discover the top- k representative locally densest subgraphs of a graph. The method involves defining a parameter-free definition of an LDS, showing that the set of LDSes in a graph can be computed in polynomial time. Trung et al. [37] observed the hierarchical structure of maximal ρ -compact subgraphs and presented verification-free approaches to improve the efficiency of finding LDSes. Ma et al. [38] proposed a convex-programming-based algorithm to the LDS problem by introducing the concept of the compact number and using the relations of compactness to the LDS problem and a specific convex program. Capitalizing on previous results [36], Samusevich et al. [39] studied the local triangle densest subgraph problem, and Xu et al. [40] proposed the locally h -clique densest subgraph problem, which extended the LDS model to general h -clique density.

However, these approaches are inherently restricted to unsigned graphs, leaving the structural complexity of signed interactions unexplored.

C. Limitations of Existing Approaches

Despite the extensive literature, existing approaches exhibit distinct structural limitations that motivate the specific design choices in this work. First, methods focusing on maximum balanced subgraphs usually yield structurally sparse results,

failing to capture the intense interactivity required to define strong antagonism. While algorithms identifying maximum balanced cliques address density, the strict enforcement of balance constraints often restricts the detected subgraphs to a trivial scale, limiting their ability to represent macro-level group conflicts. Second, approaches for polarized community detection do not strictly adhere to structural balance theory, and they often fail to distinguish between cooperation and conflict, whereas identifying substantial conflicts between distinct groups is often of greater analytical value. Furthermore, traditional densest subgraph discovery focuses solely on edge density by treating positive and negative edges identically. This neutrality prevents the distinction between a cooperative team and a hostile battlefield. These collective shortcomings necessitate a unified framework capable of simultaneously ensuring structural compactness, accommodating near-balance for scalability, and explicitly maximizing the density of conflicting edges between opposing groups.

III. PRELIMINARIES

Given an undirected and signed graph $G = (V, E)$, where V is the set of vertices and $E = E^+ \cup E^-$ is the set of signed edges such that E^+ is the set of positive edges and E^- is the set of negative edges, indicating the positive and negative relationship between the two vertices in a graph, respectively. Given a subset $S \subseteq V$, $G[S] = (S, E(S))$ is the subgraph induced by S , and $E(S) = E(G) \cap (S \times S)$. A pattern (also known as a motif) [41], [42], [43] is a small connected subgraph that appears frequently in a larger graph, which can be considered as a basic module. We use $\psi_{hx}(V_{\psi_{hx}}, E_{\psi_{hx}})$ to denote a particular kind of pattern x with h vertices in signed graphs and $\Psi_{hx}(G)$ is the collection of the hx -patterns in G . The index x is a size-specific identifier for topological structures and implies no cross-size structural consistency. Fig. 2 shows an example of signed patterns. $d_{\psi_{hx}}(G) = \frac{|\Psi_{hx}(G)|}{|V|}$ denotes the hx -pattern density of G . $deg_G(v, \psi_{hx})$ is the hx -pattern degree of v in G , i.e., the number of hx -patterns containing v . Table I summarizes the main notations used in this paper.

Definition 1 (Balanced graph [5]): Given a signed graph $G = (V, E^+, E^-)$, G is balanced if its vertex set V can be partitioned into two disjoint subsets V_L and V_R (i.e., $V_L \cup V_R = V$ and $V_L \cap V_R = \emptyset$), such that all edges between vertices in the same subset are positive and all edges between vertices from different subsets are negative.

Given two subsets V_L and V_R , the edge set that across the two subsets is E_{LR} . According to the definition of balanced graph, all signed subgraphs can be classified into two categories: the balanced subgraphs and the unbalanced subgraphs. For example, in Fig. 2, the four types of signed triangles can be classified into two categories: balanced triangles ($\Psi_{3b}^+ = \Psi_{3b}^1 \cup \Psi_{3b}^2$) and unbalanced triangles ($\Psi_{3b}^- = \Psi_{3b}^3 \cup \Psi_{3b}^4$); the eleven types of signed 4-cliques can also be classified into two categories: balanced 4-cliques ($\Psi_{4f}^+ = \Psi_{4f}^1 \cup \dots \cup \Psi_{4f}^3$) and unbalanced 4-cliques ($\Psi_{4f}^- = \Psi_{4f}^4 \cup \dots \cup \Psi_{4f}^{11}$).

In a signed graph, complete balance requires that every cycle in the graph satisfies the balance condition, meaning that each

cycle has an even number of negative edges. While this condition represents an ideal state for the network, it is difficult to realize in real-world systems. Conflicts and contradictions are common, making it nearly impossible for every local structure to meet the strict balance condition.

In addition to the balance measure, the concept of *polarity* is proposed in [18]: $POL(S) = \frac{2}{|V_L|+|V_R|} \cdot (|E_L^+| + |E_R^+| - |E_L^-| - |E_R^-| + |E_{LR}^-| - |E_{LR}^+|)$. Polarity also implicitly expresses balance. It encourages the existence of balanced edges, and penalizes the presence of unbalanced edges. For graphs with the same number of vertices and edges, the balanced graph has the largest polarity. The intra-community positive edges and inter-community negative edges can be called effective edges. Niu et al. [19] proposed that the polarity can not fully express polarization, because the polarity measure can be high even when one of the communities is empty, and they proposed the concept of *dichotomy* [19]: $DIC(S) = POL(S) \cdot \frac{|E_L^+|+|E_R^+|}{\binom{|V_L|+|V_R|}{2}} \cdot \frac{\min(|V_L|, |V_R|)}{\max(|V_L|, |V_R|)}$. Dichotomy considered the ratio of the community sizes, but the dichotomy measure focuses on the intra-community relationship, making it hard to tell whether polarization is mainly caused by intra-community identification or inter-community conflict.

Neither balance, polarity, nor dichotomy measures can indicate the intensity of conflict, because any subgraph with a large number of intra-community positive edges has high measure values, so they are ineffective for the goal of identifying and preventing conflict. The intensity of the conflict, i.e., antagonism, is important for conflict detection in signed graphs. At present, there is not a unified definition of antagonism. To address the challenge of identifying antagonistic near-balanced communities in signed networks, we propose **Antagonism**, a metric that quantifies the quality of a given subgraph S from three dimensions: **polarity, internal cohesion, and external antagonistic normalized density**, as follows:

$$ANT(S) = POL(S) \cdot \frac{|E(S)|}{\binom{|S|}{2}} \cdot \frac{|E_{LR}^-|}{\left(\frac{|V_L|+|V_R|}{2}\right)^2} \quad (1)$$

Polarity, i.e., $POL(S)$, quantifies the balance of S , which emphasizes the density of effective edges while penalizing the density of intra-community negative edges and inter-community positive edges.

Internal cohesion, i.e., $\frac{|E(S)|}{\binom{|S|}{2}}$, evaluates the cohesion of subgraph by measuring the edge density in the subgraph, aligning with the principle that antagonism requires cohesion within the subgraphs.

External antagonistic normalized density, i.e., $\frac{|E_{LR}^-|}{\left(\frac{|V_L|+|V_R|}{2}\right)^2}$, measures the intensity of antagonistic interactions between the communities. Intuitively, external antagonistic density should be the proportion of the number of negative edges between the communities to the potential maximum number of edges between the communities ($|V_L| \cdot |V_R|$) to reflect the intensity of conflict. However, this introduces a bias toward communities with large size disparities. For example, if $|V_L| = 100$, $|V_R| = 10$, $|V_L| \cdot |V_R| = 1000$, whereas for communities with same size ($|V_L| = 55$, $|V_R| = 55$), $|V_L| \cdot |V_R| = 3025$. This results in

higher antagonistic density values for communities with larger size of differences, even if their actual negative edge density is low. To resolve this, we normalize the denominator by using the square of the geometric mean of the community sizes to eliminate the size bias. The normalized metric prioritizes true conflict intensity rather than being skewed by the community size differences. Then we define the main problem as follows.

Integrating these three dimensions prevents degenerate solutions inherent to single-objective optimization. Polarity mathematically scales with average degree; maximizing it alone introduces a size bias, favoring large, sparse structures over compact cores. To mitigate this, internal cohesion serves as a topological regularizer based on edge density (the ratio of actual to possible edges), which penalizes sparsity and enforces structural compactness. Finally, external antagonistic normalized density imposes an interaction constraint to guarantee that the detected subgraphs exhibit intense adversarial interactions. The multiplicative formulation ensures that optimal subgraphs are simultaneously structurally balanced, topologically dense, and antagonistically active.

Definition 2 (Antagonistic dense subgraph Problem): Given a signed undirected graph G , the antagonistic dense subgraph problem is to find a subgraph with the largest antagonism.

Maximizing polarity is proved to be NP-hard [18], and the antagonistic dense subgraph problem is also NP-hard. In the following sections, we propose algorithms to find subgraphs with high antagonism.

IV. ANALYSIS

In this section, we establish the theoretical framework for mining antagonistic near-balanced dense subgraphs. We begin by defining antagonistic balanced patterns as the fundamental structural units that capture local conflict. To quantify the structural density of these patterns, we introduce the concept of hx -pattern compactness. Finally, we formally define the locally hx -pattern densest subgraph and the corresponding optimization problem, establishing a tractable computational proxy for identifying distinct regions with high antagonism. Specifically, maximizing antagonism corresponds to identifying antagonistic near-balanced dense subgraphs. Niu et al. [19] proposed that balanced triangles are typically closer to each other in the real networks than expected, and ψ_{3b}^2 contributes more to polarity than ψ_{3b}^1 . We define an antagonistic balanced pattern as:

Definition 3 (Antagonistic balanced pattern): A pattern is antagonistic balanced if it is a balanced pattern with at least one negative edge.

The antagonistic balanced patterns, such as ψ_{3b}^2 , ψ_{4a}^{2-4} , ψ_{4f}^{2-3} , are basic elements that embodies the antagonism in a balanced subgraph. This is due to the fact that a vertex is generally involved in multiple overlapping antagonistic situations, for instance, a person may be involved in conflicts as favoring different policies, supporting different idols, etc. The identification of antagonistic near-balanced dense subgraphs holds significant importance, as it relaxes the requirement of complete balance or complete connectivity within the graph, and allows for a certain degree of unbalance or missing connections while still maintaining a high level of connectivity, near-balance, and

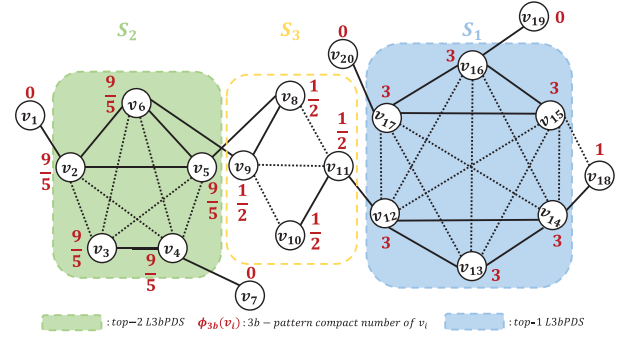


Fig. 3. An example of the locally hx -pattern densest subgraph.

antagonism. Without loss of generality, in this paper, we use simplified symbol hx -pattern to represent **all the antagonistic balanced patterns of the unsigned pattern x with h vertices.**

Given the importance of detecting large antagonistic near-balanced dense subgraphs, the hx -pattern densest subgraph (PDS) problem that finds antagonistic near-balanced dense subgraphs formed by overlapping antagonistic balanced patterns has attracted our attention. By finding the subgraph with the highest density of antagonistic balanced patterns, PDS uncovers the highly-connected component that exhibits strong internal interactions and antagonistic. Whereas, in the context of the real world, the discovery of a single PDS offers limited insights. Listing the top- k PDSes is desired, but due to the substantial overlap inherent in hx -patterns, the vanilla top- k PDSes may refer to the same dense region, still providing limited structural insights. Therefore, detecting the top- k non-overlapping, locally maximal, dense, and compact, i.e., locally hx -pattern densest subgraphs (L hx PDS) attracts great interest. We believe that the L hx PDS problem can find subgraphs with high antagonism. For example, Fig. 3 shows an example of the locally antagonistic balanced pattern densest subgraphs. According to the definition of antagonism, the L $3b$ PDSes in Fig. 3 are indeed subgraphs with high antagonism in the graph, which is consistent with our analysis.

A densest subgraph in a local region not only means that such a subgraph is not included in any other denser subgraph, but also requires the inner density to be compact and evenly distributed. Qin et al. [36] proposed the concept of ρ -compact, which gives a reasonable definition of locally densest subgraph. A graph G is ρ -compact if at least $\rho \times |S|$ edges are removed when removing any subset S from G . Considering the hx -pattern density in a graph, we define an hx -pattern ρ -compact graph as:

Definition 4 (hx -pattern ρ -compact): A graph G is hx -pattern ρ -compact if G is connected, and removing any subset of vertices $S \subseteq V$ will result in the removal of at least $\rho \times |S|$ hx -patterns in G , where ρ is a non-negative real number.

If G is hx -pattern ρ -compact, then the hx -pattern degree of each vertex in G is at least $\lceil \rho \rceil$, because removing any vertex will remove at least ρ hx -patterns. Besides, the hx -pattern density of an hx -pattern ρ -compact graph is at least ρ . For any $\hat{\rho} > \rho$, an hx -pattern $\hat{\rho}$ -compact graph is also an hx -pattern ρ -compact graph, so we define the **hx -pattern compactness** of a graph G as the largest ρ such that G is hx -pattern ρ -compact. A subgraph

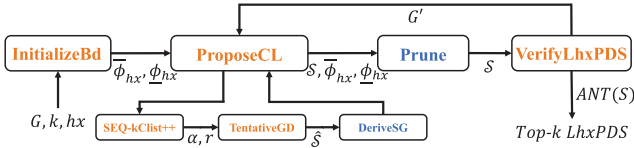


Fig. 4. Flow diagram of IPPV-S (all orange parts are our contribution and innovation for this problem, and the blue parts are extensions of existing methods).

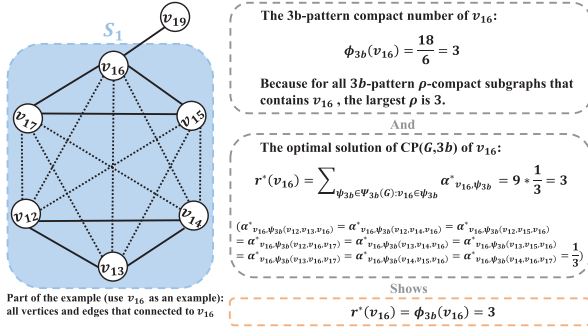


Fig. 5. An example of the relationship between $r^*(u)$ and $\phi_{hx}(u)$.

$G[S]$ of G is a **maximal hx -pattern ρ -compact subgraph** if none of the supergraphs of $G[S]$ is hx -pattern ρ -compact.

Proposition 1: If a graph G has hx -pattern density $d_{\psi_{hx}}(G)$, then the hx -pattern compactness of the graph is at most $d_{\psi_{hx}}(G)$, i.e., $\rho \leq d_{\psi_{hx}}(G)$.

Proof: Suppose the compactness of G is higher than $d_{\psi_{hx}}(G)$, then removing all vertices in G will result in the removal of more than $d_{\psi_{hx}}(G) \times |V|$ hx -patterns, which means that the hx -pattern density of G must be higher than $d_{\psi_{hx}}(G)$, which contradicts the fact that the hx -pattern density of G is $d_{\psi_{hx}}(G)$. $\square$

Proposition 1 clarifies that the hx -pattern compactness of a graph cannot be greater than $d_{\psi_{hx}}(G)$. We are then interested in finding the locally hx -pattern dense and compact subgraph $G[S]$ in G . We formally define a locally hx -pattern densest subgraph as follows.

Definition 5 (Locally hx -pattern densest subgraph): A subgraph $G[S]$ of G is a locally hx -pattern densest subgraph (L hx PDS) of G if the following two conditions hold:

- 1) $G[S]$ is an hx -pattern $d_{\psi_{hx}}(G[S])$ -compact subgraph;
- 2) There does not exist a supergraph $G[S']$ of $G[S]$ ($S' \supsetneq S$), such that $G[S']$ is also hx -pattern $d_{\psi_{hx}}(G[S])$ -compact.

Proposition 2 (Disjoint property [40]): Suppose $G[S]$ and $G[S']$ are two L hx PDSes in G , we have $S \cap S' = \emptyset$.

Proposition 2 proves that all L hx PDSes in a graph G are pairwise disjoint. Therefore, the number of L hx PDSes of G is bounded by $|V|$, and the L hx PDSes can be used to identify all the non-overlapping antagonistic balanced pattern dense regions of a graph. Most applications in the real world usually require finding the top- k dense regions of a graph [36], so we focus on finding the top- k L hx PDSes with the largest antagonism. When k is large enough, all L hx PDSes can be found. We formulate the problem as follows.

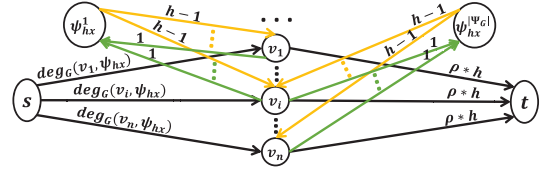


Fig. 6. The flow network of $\text{DeriveCompact}(G, \rho, \emptyset)$.

Definition 6 (Locally hx -pattern densest subgraph Problem (L hx PDS Problem)): Given a signed graph G , an integer h , a pattern x , and an integer k , the locally hx -pattern densest subgraph problem is to compute the top- k L hx PDSes ranked by the antagonism.

As shown in Fig. 3, we use S_1, S_2 , and S_3 to represent $\{v_{12}, \dots, v_{17}\}$, $\{v_2, \dots, v_6\}$, and $\{v_8, \dots, v_{11}\}$. When $hx = 3b$, the top-1 and top-2L $3b$ PDSes are $G[S_1]$ and $G[S_2]$, which have a $3b$ -pattern density of 3 and $\frac{9}{5}$, since there are eighteen and nine $3b$ -patterns in $G[S_1]$ and $G[S_2]$. Besides, $\text{ANT}(S_1) = 5 \cdot 1 \cdot 1 = 5$, $\text{ANT}(S_2) = 4 \cdot 1 \cdot \frac{24}{25} = \frac{96}{25}$.

The antagonistic dense subgraph problem is NP-hard. We address this by utilizing the L hx PDS problem as a structural proxy, which transforms the intractable global optimization into a local density search. This approximation strategy is motivated by the structural alignment with the **antagonism** measure. The antagonistic balanced patterns serve as atomic units that capture **polarity** and **external antagonistic normalized density** (via mandatory negative edges), while the locally densest constraint promotes high **internal cohesion**. Therefore, maximizing the local density of these patterns constructs a candidate space that effectively targets high-antagonism regions, allowing the final ranking step to identify near-optimal solutions.

V. L hx PDS DISCOVERY

To efficiently discover locally antagonistic balanced pattern densest subgraphs, we propose an algorithmic pipeline named IPPV-S. As illustrated in Fig. 4, the framework follows a “iterative propose-prune-and-verify” paradigm. While the concept of hx -pattern compactness characterizes the cohesiveness of a subgraph, a single vertex may be contained in various subgraphs with different compactness levels. Therefore, to quantify the maximal compactness of each vertex, we introduce the concept of hx -pattern compact number.

Definition 7 (hx -pattern compact number): Given a graph $G = (V, E)$, for each vertex $u \in V$, the hx -pattern compact number of u is the largest ρ such that u is contained in an hx -pattern ρ -compact subgraph of G , denoted by $\phi_{hx}(u)$.

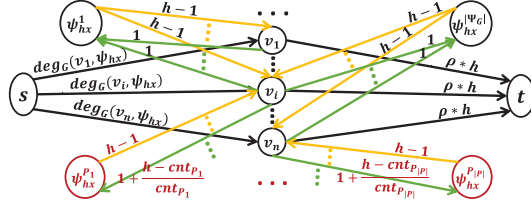
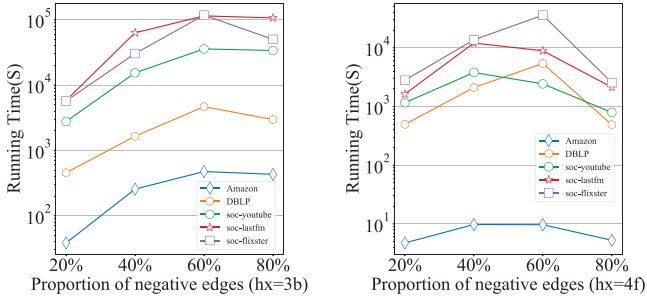
We can prove the relationship between the L hx PDS and the hx -pattern compact numbers of vertices within it.

Theorem 1: Given an L hx PDS $G[S]$ in G , for each vertex $u \in S$, the hx -pattern compact number of u is equal to the hx -pattern density of $G[S]$, i.e., $\phi_{hx}(u) = d_{\psi_{hx}}(G[S])$.

Proof: As $G[S]$ is a maximal hx -pattern $d_{\psi_{hx}}(G[S])$ -compact subgraph, for each $u \in S$, there exists no other subgraph $G[S']$ containing u such that $G[S']$ is an hx -pattern ρ -compact subgraph with $\rho > d_{\psi_{hx}}(G[S])$. We prove the claim by contradiction. Suppose $G[S']$ is an hx -pattern

TABLE II
 STATISTICS OF THE REAL-WORLD SIGNED GRAPHS

Name	Abbr.	$ V(G) $	$ E(G) $	$ E^- / E $	triangles	Ψ_{3b}^2	4-cliques	Ψ_{4f}^{2-3}
soc-sign-bitcoinalpha	BA	3,783	14,124	0.08	22,153	1,499	30,948	1,386
soc-sign-bitcoinot	BO	5,881	21,491	0.14	33,466	5,025	48,312	7,920
wikiElec-ElecB3	EB	7,118	100,820	0.22	612,469	56,843	2,125,557	57,074
soc-redditHyperlinks-body	RH	35,776	124,330	0.07	406,391	21,142	1,761,565	26,042
soc-sign-Slashdot	SS	77,350	468,554	0.24	548,054	74,737	1,968,248	63,129
wikiconflict	WC	116,717	2,026,646	0.63	13,831,236	3,958,487	63,142,975	6,837,456
soc-sign-epinions	EP	131,580	711,210	0.17	4,910,076	416,882	58,567,924	2,063,126


 Fig. 7. The flow network of $\text{DeriveCompact}(G, \rho, P)$.

 Fig. 8. Running time on five real-world unsigned datasets with varying proportions of negative edges for patterns $hx = 3b$ and $hx = 4f$.

ρ -compact subgraph with $\rho > d_{\psi_{hx}}(G[S])$ and $u \in S'$, we have $d_{\psi_{hx}}(G[S']) \geq \rho > d_{\psi_{hx}}(G[S])$. First, $S' \subseteq S$, because $G[S]$ is a maximal hx -pattern $d_{\psi_{hx}}(G[S])$ -compact subgraph and $S' \cap S \neq \emptyset$. If we remove $U = S \setminus S'$ from $G[S]$, the number of hx -patterns removed is $|\Psi_{hx}(G[S])| - |\Psi_{hx}(G[S'])| = d_{\psi_{hx}}(G[S]) \times |S| - d_{\psi_{hx}}(G[S']) \times |S'| < d_{\psi_{hx}}(G[S]) \times (|S| - |S'|) = d_{\psi_{hx}}(G[S]) \times |U|$. This contradicts that $G[S]$ is hx -pattern $d_{\psi_{hx}}(G[S])$ -compact. Hence, $d_{\psi_{hx}}(G[S])$ is the hx -pattern compact number of all vertices in S . $\square$

Based on Theorem 1, once we get the hx -pattern compact number of each vertex in G , we can arrange the vertices in descending order based on the hx -pattern compact number and then check whether the vertices with the same hx -pattern compact number satisfy the definition of L hx PDS to obtain top- k L hx PDSes. For example, in Fig. 3, we list the antagonistic balanced $3b$ -pattern compact numbers of all vertices of G . Obviously, $G[S_1]$ and $G[S_2]$ are both L $3b$ PDSes.

However, computing the hx -pattern compact numbers directly is difficult. To address this challenge, IPPV-s implements the aforementioned “iterative propose-prune-and-verify” strategy by jointly optimizing the compact number bounds and verifying candidates.

Fig. 4 shows the flow diagram of IPPV-s. It has four main parts: **1)** calculate the initial bounds of the hx -pattern compact numbers of vertices; **2)** iteratively propose all L hx PDS candidates (generating approximate hx -pattern compact numbers; decomposing the graph tentatively; grouping vertices

Algorithm 1: Bound Initialization: InitializeBd:

Input: $G = (V, E), h$
Output: $\bar{\phi}_{hx}, \underline{\phi}_{hx}$
1 **foreach** $u \in V$ **do** compute $core_G(u, \psi_{hx})$;
2 **foreach** $u \in V$ **do**
3 $\bar{\phi}_{hx}(u) \leftarrow core_G(u, \psi_{hx})$; $\underline{\phi}_{hx}(u) \leftarrow \frac{core_G(u, \psi_{hx})}{h}$;
4 **return** $\bar{\phi}_{hx}, \underline{\phi}_{hx}$;

and tightening bounds); **3)** prune ineligible vertices; **4)** verify the locally densest property of all candidates to find top- k L hx PDSes ranked by the antagonism.

A. Initial hx -Pattern Compact Number Bounds

In order to derive L hx PDS candidates, we first give initial upper and lower bounds of hx -pattern compact number $\phi_{hx}(u)$. Specifically, we denote $\bar{\phi}_{hx}(u)$ and $\underline{\phi}_{hx}(u)$ as the upper and lower bound of $\phi_{hx}(u)$ in G . We use (k, ψ_{hx}) -core [28], which is a cohesive subgraph model, to compute the initial bounds. Given a graph G , the (k, ψ_{hx}) -core is the largest subgraph of G , in which the hx -pattern degree of each vertex is at least k . The hx -pattern-core number of a vertex $u \in V$, denoted by $core_G(u, \psi_{hx})$, is the highest k of (k, ψ_{hx}) -core containing u .

Proposition 3: $\phi_{hx}(u)$ has the following relations to the hx -pattern-core number $core_G(u, \psi_{hx})$:

- 1) A (k, ψ_{hx}) -core subgraph is hx -pattern $\frac{k}{h}$ -compact. Thus, for any $u \in V$, $\underline{\phi}_{hx}(u)$ can be assigned as $\frac{core_G(u, \psi_{hx})}{h}$;
- 2) If $G[S]$ is an L hx PDS of G , for all $u \in S$, $core_G(u, \psi_{hx}) \geq d_{\psi_{hx}}(G[S])$. Thus, for any $u \in V$, $\bar{\phi}_{hx}(u)$ can be assigned as $core_G(u, \psi_{hx})$.

Proof: Any vertex in a (k, ψ_{hx}) -core subgraph is contained in at least k hx -patterns. By removal of any subset S from the (k, ψ_{hx}) -core, at least $\frac{k}{h} \times |S|$ hx -patterns would be removed. For any $u \in V$, there is an hx -pattern $\frac{core_G(u, \psi_{hx})}{h}$ -compact subgraph of G that contains u , then $\frac{core_G(u, \psi_{hx})}{h}$ is a lower bound of $\phi_{hx}(u)$. The second relation can be obtained from the fact that an L hx PDS $G[S]$ in a graph G is a $(\lceil d_{\psi_{hx}}(G[S]) \rceil, \psi_{hx})$ -core subgraph of G . For any $u \in V$, if an L hx PDS contains u , then $core_G(u, \psi_{hx})$ is an upper bound of $\phi_{hx}(u)$. $\square$

According to Proposition 3, we can get the initial bounds of $\phi_{hx}(u)$ of G (Lines 2-3) by Algorithm 1.

B. Candidate L hx PDS Proposal

The initial upper and lower bounds for hx -pattern compact numbers are relatively loose. In this section, we focus on how to tighten the bounds and propose L hx PDS candidates.

Overall algorithm for candidate L hx PDS proposal: The overall candidate L hx PDS proposal algorithm is given in Algorithm 2. Approximate hx -pattern compact number is calculated via SEQ-kClist++ (Line 1); the preliminary partition of G and recalculated values are obtained via TentativeGD (Line 2); tighter upper and lower bounds for hx -pattern compact numbers and the further partition of G (stable hx -pattern group) are calculated via DeriveSG (Line 3). The sub-procedures introduce each of the above functions (Lines 5-33).

Algorithm 2: Candidate L hx PDS Proposal: ProposeCL:

Input: $G = (V, E)$, number of iterations T , $\bar{\phi}_{hx}, \underline{\phi}_{hx}$
Output: $\mathcal{S}, \bar{\phi}_{hx}, \underline{\phi}_{hx}$

```

1   $(\alpha, r) \leftarrow \text{SEQ-kClist++}(G, T)$ ;
2   $\hat{\mathcal{S}}, \alpha, r \leftarrow \text{TentativeGD}(G, \alpha, r)$ ;
3   $\mathcal{S}, \bar{\phi}_{hx}, \underline{\phi}_{hx} \leftarrow \text{DeriveSG}(\hat{\mathcal{S}}, \alpha, r, \bar{\phi}_{hx}, \underline{\phi}_{hx})$ ;
4  return  $\mathcal{S}, \bar{\phi}_{hx}, \underline{\phi}_{hx}$ ;
5  Procedure SEQ-kClist++( $G, T$ )
6    foreach  $hx\text{-pattern } \psi_{hx} \text{ in } G$  do  $\alpha_{u, \psi_{hx}} \leftarrow \frac{1}{h}, \forall u \in V_{\psi_{hx}}$ ;
7    foreach  $u \in V$  do  $r(u) \leftarrow \sum_{\psi_{hx} \in \Psi_{hx}(G): u \in \psi_{hx}} \alpha_{u, \psi_{hx}}$ ;
8    foreach  $\text{iteration } t = 1, \dots, T$  do
9       $\gamma_t \leftarrow \frac{1}{t+1}; \alpha \leftarrow (1 - \gamma_t) * \alpha; r \leftarrow (1 - \gamma_t) * r$ ;
10     foreach  $hx\text{-pattern } \psi_{hx}$  do
11        $v_{min} \leftarrow \text{argmin}_{v \in \psi_{hx}} r(v)$ ;
12        $\alpha_{v_{min}, \psi_{hx}} \leftarrow \alpha_{v_{min}, \psi_{hx}} + \gamma_t$ ;
13        $r(v_{min}) \leftarrow r(v_{min}) + \gamma_t$ ;
14   return  $(\alpha, r)$ ;
15  Procedure TentativeGD( $G, \alpha, r$ )
16   sort vertices in  $V$  in descending order according to  $r$ ;
17    $P \leftarrow \{p | p = \text{argmax}_{p \leq q \leq n} d_{\psi_{hx}}(G[V[1:q]])\}$ ;
18    $\hat{\mathcal{S}} \leftarrow \text{partition } V \text{ according to } P$ ;
19   foreach  $\psi_{hx} \in \Psi_{hx}(G)$  do
20      $p \leftarrow \max \{1 \leq i \leq l : \psi_{hx} \cap \hat{\mathcal{S}}_i \neq \emptyset\}$ ;
21      $s \leftarrow \sum_{u \in \psi_{hx} \setminus \hat{\mathcal{S}}_p} \alpha_{u, \psi_{hx}}$ ;
22      $\forall u \in \psi_{hx} \setminus \hat{\mathcal{S}}_p, \alpha_{u, \psi_{hx}} \leftarrow 0$ ;
23      $\forall u \in \psi_{hx} \cap \hat{\mathcal{S}}_p, \alpha_{u, \psi_{hx}} \leftarrow \alpha_{u, \psi_{hx}} + \frac{s}{|\psi_{hx} \cap \hat{\mathcal{S}}_p|}$ ;
24   foreach  $u \in V$  do  $r(u) \leftarrow \sum_{\psi_{hx} \in \Psi_{hx}(G): u \in \psi_{hx}} \alpha_{u, \psi_{hx}}$ ;
25   return  $\hat{\mathcal{S}}, \alpha, r$ ;
26  Procedure DeriveSG( $\hat{\mathcal{S}}, \alpha, r, \bar{\phi}_{hx}, \underline{\phi}_{hx}$ )
27   while  $\hat{\mathcal{S}}$  is not empty do
28      $S' \leftarrow \text{pop out the first candidate from } \hat{\mathcal{S}}; S \leftarrow S \cup S'$ ;
29     if  $S$  is a stable  $hx\text{-pattern group}$  then put  $S$  into  $\mathcal{S}$ ;
30      $S \leftarrow \emptyset$ ;
31   foreach  $S \in \mathcal{S}$  do
32     foreach  $u \in S$  do
33        $\bar{\phi}_{hx}(u) \leftarrow \min\{\bar{\phi}_{hx}(u), \max_{v \in S} r(v)\}$ ;
34        $\underline{\phi}_{hx}(u) \leftarrow \max\{\underline{\phi}_{hx}(u), \min_{v \in S} r(v)\}$ ;
35   return  $\mathcal{S}, \bar{\phi}_{hx}, \underline{\phi}_{hx}$ ;

```

1) *Generate Approximate hx -Pattern Compact Number:* Inspired by a classical convex programming [30], [38], we propose a convex programming (CP) for finding the diminishingly- hx -pattern-dense decomposition, and prove that the optimal solution of our convex programming is exactly the hx -pattern compact number of a graph G .

Intuitively, the aim of $\text{CP}(G, hx)$ is that each hx -pattern $\psi_{hx} \in \Psi_{hx}(G)$ tries to distribute its unit weight among its h vertices such that the sum of the weight received by the vertices are as even as possible. We use $\alpha_{u, \psi_{hx}}$ to represent the weight assigned to u from hx -pattern ψ_{hx} and $r(u)$ to denote the sum of the weights assigned to u from hx -patterns that contain u . This intuition suggests that we can consider the objective function: $Q_{G, hx}(\alpha) := \sum_{u \in V} r(u)^2$, in which $r(u) = \sum_{\psi_{hx} \in \Psi_{hx}(G): u \in \psi_{hx}} \alpha_{u, \psi_{hx}}$, for all $u \in V$. The convex programming is:

$$\text{CP}(G, hx) := \min \{Q_{G, hx}(\alpha) : \alpha \in \mathcal{D}(G, hx)\},$$

where the domain is:

$$\mathcal{D}(G, hx) := \left\{ \alpha \in \prod_{\psi_{hx} \in \Psi_{hx}(G)} \mathbb{R}_+^{\psi_{hx}} : \forall \psi_{hx} \in \Psi_{hx}(G), \sum_{u \in \psi_{hx}} \alpha_{u, \psi_{hx}} = 1 \right\}.$$

Here, we demonstrate that the hx -pattern compact numbers can be derived from the optimal solution of $\text{CP}(G, hx)$.

Theorem 2: Let (α^*, r^*) be an optimal solution to $\text{CP}(G, hx)$. Then, $\forall u \in V$, $\phi_{hx}(u) = r^*(u)$, i.e., each $r^*(u)$ in r^* is exactly the hx -pattern compact number of u .

Proof: For any vertex $u \in V$, let $S^+ = \{v \in V | r^*(v) > r^*(u)\}$, $S^= = \{v \in V | r^*(v) = r^*(u)\}$, $S^- = \{v \in V | r^*(v) < r^*(u)\}$, $u \in S^=$. $S^{+=}$ denotes the vertices that are contained by hx -patterns that including vertices both in S^+ and $S^=$. We prove $G[S^+ \cup S^=]$ is an hx -pattern $r^*(u)$ -compact subgraph. First, removing $S^=$ from $G[S^+ \cup S^=]$ will result in the removal of $r^*(u) \times |S^=|$ patterns in $G[S^+ \cup S^=]$. We know that for all $(v, w) \in E \cap (S^+ \times S^=)$, $r^*(v) > r^*(w)$ and $\alpha_{v, \psi_{hx}}(v, w \in \psi_{hx}) = 0$. Otherwise, if there exists $(v, w) \in E \cap (S^+ \times S^=)$ such that $\alpha_{v, \psi_{hx}}(v, w \in \psi_{hx}) > 0$, there exists $r^*(v) - r^*(w) > \epsilon > 0$. We can reduce $\alpha_{v, \psi_{hx}}(v, w \in \psi_{hx})$ by ϵ and increase $\alpha_{w, \psi_{hx}}(v, w \in \psi_{hx})$ by ϵ , and the objective function be decreased by $2\epsilon(r^*(v) - r^*(w) - \epsilon)$, which contradicts the optimality of r^* . Similarly, we can prove that for all $(v, w) \in E \cap (S^= \times S^-)$, $r^*(v) > r^*(w)$ and $\alpha_{v, \psi_{hx}}(v, w \in \psi_{hx}) = 0$. Therefore, $r^*(u) \times |S^=| = \sum_{\psi_{hx} \in \Psi_{hx}(G): v \in S^=, v \in \psi_{hx}} \alpha_{v, \psi_{hx}} = |\psi_{hx}(G[S^=]) \cup \psi_{hx}(G[S^{+=}])|$. $r^*(u) \times |S^=|$ is exactly the number of hx -patterns to be removed when removing $S^=$ from $G[S^+ \cup S^=]$. Meanwhile, for any $S' \subseteq S^+ \cup S^=$, we have that $r^*(u) \times |S'| \leq \sum_{\psi_{hx} \in \Psi_{hx}(G): v \in S', v \in \psi_{hx}} \alpha_{v, \psi_{hx}} \leq \sum_{\psi_{hx} \in \psi_{hx}(G[S^+ \cup S^=]): v \in S', v \in \psi_{hx}} 1$, which means removing any $S' \subseteq S^+ \cup S^=$ from $G[S^+ \cup S^=]$ will result in the removal of at least $r^*(u) \times |S'|$ hx -patterns. Therefore, $G[S^+ \cup S^=]$ is an hx -pattern $r^*(u)$ -compact subgraph. Analogously, we can prove that for any other subset S'' containing u , $G[S'']$ is an hx -pattern ρ -compact subgraph, where $\rho \leq r^*(u)$, by contradiction. So $r^*(u)$ is the largest ρ such that u is contained in an hx -pattern ρ -compact subgraph of G , which is exactly the hx -pattern compact number of u . $\square$

Consider the $\text{CP}(G, 3b)$ for G in Fig. 3, we use v_{16} as an example, shown in Fig. 5. The antagonistic balanced $3b$ -pattern compact number of v_{16} is 3, and the optimal solution $r^*(v_{16})$ value is also 3. It is clear that for each $u \in V$, $r^*(u)$ is exactly $\phi_{hx}(u)$. According to Theorem 2, the L hx PDS problem is equivalent to $\text{CP}(G, hx)$. $\text{CP}(G, hx)$ is a convex quadratic programming problem, which has polynomial time solutions [44], [45]. Therefore, the L hx PDS problem is polynomial-time solvable, and we propose Theorem 3.

Theorem 3: The L hx PDS discovery problem can be solved in polynomial time for any given positive integer h .

However, $\text{CP}(G, hx)$ is a problem with $|\Psi_{hx}|$ equality constraints and $n \cdot |\Psi_{hx}|$ variables. The time complexity of the classical polynomial-time solution of $\text{CP}(G, hx)$ is $O((n \cdot |\Psi_{hx}|)^3)$.

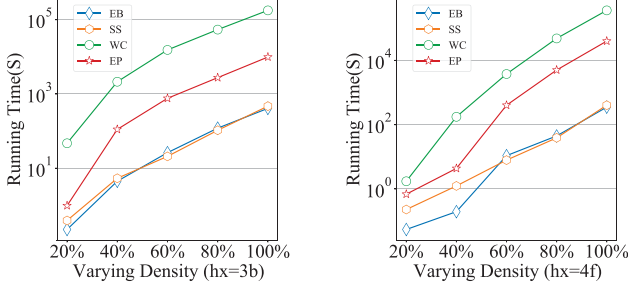


Fig. 9. Scalability evaluation: Running time on synthetically generated graphs with varying densities.

L [44], which requires long running time. Therefore, exactly attaining the (α^*, r^*) is difficult, so we use the approximate solution (α, r) of $CP(G, hx)$ to tighten the hx -pattern compact bounds. SEQ-kClist++ [35] is efficient for approximately calculating $\alpha_{u, \psi_{hx}}$ for each hx -pattern ψ_{hx} , $u \in \psi_{hx}$, as well as $r(u)$ for each vertex u . All $\alpha_{u, \psi_{hx}}$ are initialized to $\frac{1}{h}$ (Line 6). $r(u)$ stores the sum over all $\alpha_{u, \psi_{hx}}$'s such that ψ_{hx} contains u (Line 7). At each iteration, α and r are modified simultaneously as follows. For each hx -pattern ψ_{hx} , we find the minimum $r(v_{min})$ among ψ_{hx} , and new values for the $\alpha_{v_{min}, \psi_{hx}}$ and the $r(v_{min})$ are computed as convex combinations (Lines 8-12).

2) *Tentative Graph Decomposition*: After getting approximate (α, r) , we can derive a graph decomposition from the given (α, r) .

Proposition 4: Given an L hx PDS $G[S]$ in G , $\forall (u, v) \in E$, if $u \in S$ and $v \in V \setminus S$, we have $\phi_{hx}(u) > \phi_{hx}(v)$.

Considering vertices adjacent to $G[S_1]$ but not in $G[S_1]$ in Fig. 3, such as v_{11} and v_{18} , their antagonistic balanced $3b$ -pattern compact numbers fulfill Proposition 4: $\phi_3(v_{11}) = \frac{1}{2} < 3$, $\phi_3(v_{18}) = 1 < 3$. Proposition 4 is helpful for choosing L hx PDSes from all subgraphs.

We then propose TentativeGD to generate tentative graph decomposition for proposing L hx PDS. The vertices in V are sorted based on r values descendingly (Line 15). The initial partition $\hat{S}$ of the graph is extracted based on the descending order (Lines 16-17). For each $\psi_{hx} \in \Psi_{hx}(G)$, if the pattern ψ_{hx} is contained in multiple vertex sets, the vertex set with the largest set index will be recorded as p , and the α value of ψ_{hx} will be redistributed to vertices in $\hat{S}_p$ (Lines 18-22). In other words, for the convenience of partition, the α value of ψ_{hx} straddling multiple vertex sets is redistributed to a vertex set with the lowest r value. Finally, the r values of all vertices in V are recalculated (Line 23).

3) *Stable hx -Pattern Group Derivation*: After getting the initial bounds of hx -pattern compact numbers and a preliminary partition of the graph, we consider obtaining tighter bounds of hx -pattern compact numbers and a further partition of the graph, to calculate L hx PDS candidates. Inspired by the stable group [38], for solving the problem in this paper, we propose the definition of the stable hx -pattern group.

Definition 8 (stable hx -pattern group): Given a feasible solution (α, r) to $CP(G, hx)$, a stable hx -pattern group with respect to (α, r) is a non-empty vertex group $S \subseteq V$ satisfying the following conditions:

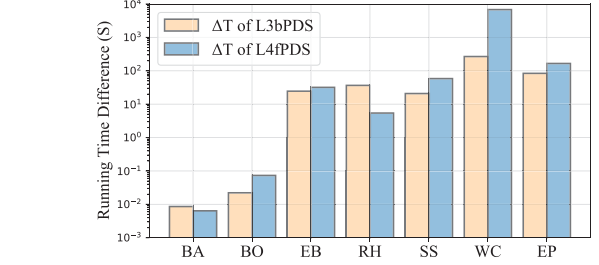


Fig. 10. Running time difference (ΔT) between the basic and the proposed fast verification algorithms across different datasets.

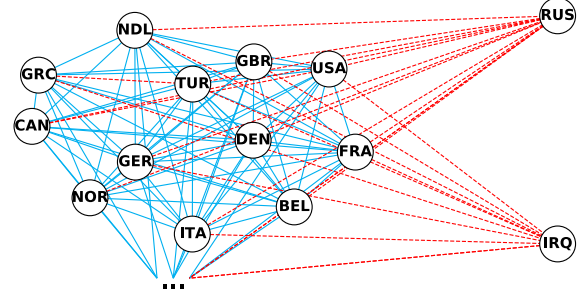


Fig. 11. Visualization of the antagonistic near-balanced dense subgraph. The ellipsis marks additional nodes in the left group omitted for clarity.

- 1) For any $v \in V \setminus S$, $r(v) > \max_{u \in S} r(u)$ or $r(v) < \min_{u \in S} r(u)$;
- 2) For any $v \in V$, if $r(v) > \max_{u \in S} r(u)$, $\forall \psi_{hx}(u, v \in \psi_{hx})$, $\alpha_{v, \psi_{hx}} = 0$;
- 3) For any $v \in V$, if $r(v) < \min_{u \in S} r(u)$, $\forall \psi_{hx}(u, v \in \psi_{hx})$, $\alpha_{u, \psi_{hx}} = 0$.

All stable hx -pattern groups are disjoint, and we use the stable hx -pattern group in our algorithm.

Theorem 4: Given a feasible solution (α, r) to $CP(G, hx)$ and a stable hx -pattern group S with respect to (α, r) , for all $v \in S$, we have that $\min_{u \in S} r(u) \leq \phi_{hx}(v) \leq \max_{u \in S} r(u)$.

Proof: According to Theorem 2, for all $u \in V$, $r^*(u) = \phi_{hx}(u)$. Suppose there exists a vertex $v \in S$ such that $r^*(v) = \phi_{hx}(v) < \min_{u \in S} r(u) \leq r(v)$. Since $\sum_{u \in V} r(u) = \sum_{u \in V} r^*(u)$, correspondingly, there must exist another vertex $w \in V$, $r^*(w) = \phi_{hx}(w) > r(w)$. The difference between $r(w)$ and $r^*(w)$ means that there exists ψ_{hx} containing both v and w , $\alpha_{v, \psi_{hx}} > 0$. Since S is a stable hx -pattern group, according to the third condition in Definition 8, $r(w) > \min_{u \in S} r(u)$. There exists $\epsilon > 0$, we can increase $r^*(v)$ by ϵ and decrease $r^*(w)$ by ϵ to decrease the value of the objective function. This contradicts that r^* is the optimal solution to $CP(G, hx)$. By the same token, for all $u \in V$, $r^*(u) = \phi_{hx}(u) \leq \max_{u \in S} r(u)$. $\square$

Based on Theorem 4, the stable hx -pattern groups can give tighter bounds of hx -pattern compact numbers, so we propose DeriveSG algorithm to derive the stable hx -pattern groups, which are our L hx PDS candidates. In DeriveSG, the subsets in $\hat{S}$ are checked one by one; if the subset is a stable hx -pattern group, it will be pushed into the set of stable hx -pattern groups S ; otherwise, in the next iteration, the current subset S will be merged with the next subset S' (Lines 26-28). Then, the upper and lower bounds of hx -pattern compact numbers are updated based on Theorem 4 (Lines 29-32).

Algorithm 3: Pruning Algorithm: Prune:

Input: $G = (V, E)$, $\mathcal{S}$, $\bar{\phi}_{hx}$, $\underline{\phi}_{hx}$
Output: $\mathcal{S}$

- 1 $G' = (V(G'), E(G')) \leftarrow G$;
- 2 **foreach** $(u, v) \in E$ **do**
- 3 **if** $\bar{\phi}_{hx}(v) < \underline{\phi}_{hx}(u)$ **then** remove v from G' ;
- 4 **foreach** $u \in V(G')$ **do** compute $core_{G'}(u, \psi_{hx})$;
- 5 **while** there exists $u \in V(G')$, $core_{G'}(u, \psi_{hx}) < \underline{\phi}_{hx}(u)$ **do**
- 6 remove u from G' ;
- 7 update hx -pattern-core numbers of vertices adjacent to u ;
- 8 **foreach** $LhxPDS$ candidate $S \in \mathcal{S}$ **do** $\mathcal{S} \leftarrow \mathcal{S} \cap V(G')$;
- 9 **return** $\mathcal{S}$;

C. Pruning for Candidate L hx PDS Derivation

We prove that the following proposition can help to prune ineligible vertices that are certainly not contained by any locally antagonistic balanced pattern densest subgraph.

Proposition 5: For any $v \in V$, v is not contained by any L hx PDS in G if either of the two conditions is satisfied.

1) If there exists $(u, v) \in E$, such that $\underline{\phi}_{hx}(u) > \bar{\phi}_{hx}(v)$, v is ineligible;

2) Let G' denote the graph after pruning all ineligible vertices in condition (1). $\bar{\phi}_{hx}^G(u)$ is the upper bound of $\phi_{hx}^G(u)$ in G . For any u in G' , if $\bar{\phi}_{hx}^G(v) < \underline{\phi}_{hx}(v)$, v is ineligible.

Proof: First, we prove condition (1). For any $u, v \in V$, $(u, v) \in E$, if $\underline{\phi}_{hx}(u) > \bar{\phi}_{hx}(v)$, then $\phi_{hx}(u) > \phi_{hx}(v)$. According to Proposition 4, v is not contained in L hx PDS, i.e., v is ineligible.

For condition (2), $\bar{\phi}_{hx}^G(u) < \underline{\phi}_{hx}(u)$ means that to form an hx -pattern $\underline{\phi}_{hx}(u)$ -compact subgraph containing u , some already pruned vertices are needed. So using the vertices in G' only cannot form an hx -pattern $\underline{\phi}_{hx}(u)$ -compact subgraph containing u . Therefore, u cannot be contained by any L hx PDS in G , i.e., v is ineligible. $\square$

According to Proposition 5, we design Pruning Rule to prune ineligible vertices by condition (1) and condition (2). An example can be seen in Fig. 3 with $hx = 3b$. v_9 and v_{11} can be pruned, because for edge (v_6, v_9) , $\underline{\phi}_{3b}(v_6) = \frac{9}{5} > \bar{\phi}_{3b}(v_9) = \frac{1}{2}$; for edge (v_{11}, v_{12}) , $\underline{\phi}_{3b}(v_{12}) = 3 > \bar{\phi}_{3b}(v_{11}) = \frac{1}{2}$. Analogously, the vertices v_1, v_7, v_{18}, v_{19} , and v_{20} are also pruned by condition (1).

We denote the graph after pruning by G' . Some vertices in G' become ineligible vertices, because any L hx PDS in G containing these vertices needs to include some already pruned vertices, which can not form an L hx PDS in G' . Therefore, we utilize condition (2). Based on Proposition 3, for any vertex $u \in V(G')$, $core_{G'}(u, \psi_{hx})$ provides an upper bound of $\phi_{hx}^G(u)$. For example, after v_9 and v_{11} are pruned, the upper bounds of $3b$ -pattern compact numbers of v_8 and v_{10} in graph G' are $\bar{\phi}_{3b}^{G'}(v_8) = \bar{\phi}_{3b}^{G'}(v_{10}) = 0 < \underline{\phi}_{3b}(v_8) = \underline{\phi}_{3b}(v_{10}) = \frac{1}{2}$. So v_8 and v_{10} are pruned using condition (2).

We propose Prune algorithm shown in Algorithm 3. G is replicated to G' for pruning (Line 1). Condition (1) is used to remove ineligible vertices in G' (Lines 2-3); after computing the $core_{G'}$ for all vertices in G' (Line 4), condition (2) is applied to further remove ineligible vertices in G' (Lines 5-7). Finally,

TABLE III
STATISTICS OF THE REAL-WORLD UNSIGNED GRAPHS

Name	$ V(G) $	$ E(G) $	$\frac{ E^- }{ E }$	triangles	Ψ_{2b}^2	4-cliques	Ψ_{4f}^{2-3}
Amazon	334,863	925,872	20% 40% 60% 80%	667,129	64,047 191,723 287,532 256,216	275,961	5,266 22,889 32,439 17,980
DBLP	317,080	1,049,866	20% 40% 60% 80%	2,224,385	213,241 639,103 960,379 857,601	16,713,192	323,924 1,377,194 1,978,754 1,128,757
soc-youtube	495,957	1,936,748	20% 40% 60% 80%	2,443,886	240,186 710,083 1,057,007 934,702	3,837,722	76,740 323,287 452,550 246,955
soc-lastfm	1,191,805	4,519,330	20% 40% 60% 80%	3,946,207	378,786 1,134,288 1,703,498 1,515,813	6,399,888	124,537 529,584 750,509 422,448
soc-flixster	2,523,386	7,918,801	20% 40% 60% 80%	7,897,122	760,199 2,272,346 3,410,920 3,039,300	23,340,258	447,539 1,919,199 2,749,724 1,557,377

the L hx PDS candidates are updated from the intersection of hx -pattern stable groups and unpruned vertex sets (Line 8).

D. L hx PDS Verification

Since candidate L hx PDSes are obtained approximately, we need to confirm whether the candidates are L hx PDSes. We need to verify: **1)** whether a candidate L hx PDS $G[S]$ is self-densest and **2)** whether $G[S]$ is a maximal hx -pattern $d_{\psi_{hx}}(G[S])$ -compact subgraph in G . We use IsDensest [35] algorithm to check whether a candidate L hx PDS $G[S]$ is self-densest. In this section, we focus on the verification of the second property, to verify whether $G[S]$ is a connected component of maximal hx -pattern $d_{\psi_{hx}}(G[S])$ -compact subgraphs in G . We design a basic verification algorithm, and we further propose a fast algorithm by reducing the scale of the flow network. The correctness of both algorithms is proved.

1) Basic Verification Algorithm: Given an L hx PDS candidate $G[S]$, we propose an innovative flow network to derive maximal hx -pattern $d_{\psi_{hx}}(G[S])$ -compact subgraph G' in G . If $G[S]$ is a connected component of G' , $G[S]$ is indeed maximal hx -pattern $d_{\psi_{hx}}(G[S])$ -compact subgraph and an L hx PDS in G ; otherwise, $G[S]$ is not an L hx PDS. The flow network $\mathcal{F}(V_{\mathcal{F}}, E_{\mathcal{F}})$ is shown in Fig. 6. The vertex set of $\mathcal{F}$ is $\{s\} \cup V \cup \Psi_{hx} \cup \{t\}$. The arc set of $\mathcal{F}$ is given as follows. For each hx -pattern ψ_{hx}^j , we add h incoming arcs of capacity 1 from the vertices which form ψ_{hx}^j , and h outgoing arcs of capacity of $h - 1$ to the same set of vertices. For each vertex $v_i \in V$, we add an incoming arc of capacity $deg_G(v_i, \psi_{hx})$ from the source vertex s , and an outgoing arc of capacity $\rho * h$ to the sink vertex t . Given a parameter ρ , we prove that the flow network in DeriveCompact can be used to derive maximal hx -pattern ρ -compact subgraphs in G in Theorem 5.

Theorem 5: If G contains maximal hx -pattern ρ -compact subgraphs, then the result returned by DeriveCompact $(G, \rho - \frac{1}{|V|^2}, \emptyset)$ is the set of all maximal hx -pattern ρ -compact subgraphs in G .

Proof: Based on Proposition 2, two L hx PDSes are disjoint. We use $G[S_1]$ to represent the union of all maximal hx -pattern ρ -compact subgraphs in G . $G[S_2]$ denotes the subgraph returned by DeriveCompact $(G, \rho - \frac{1}{|V|^2}, \emptyset)$, which is the largest subgraph in G with maximum $|\psi_{hx}(G[S_2])| - \rho \times |S_2|$

Algorithm 4: Basic L hx PDS Verification Algorithm:

Input: $G(V, E), S$
Output: VerifyL hx PDS

```

1  $\rho \leftarrow d_{\psi_{hx}}(G[S]), \text{VerifyL}hx\text{PDS} \leftarrow \text{True};$ 
2  $G' \leftarrow \text{DeriveCompact}(G, \rho - \frac{1}{|V|^2}, \emptyset);$ 
3 return  $G[S]$  is a connected component in  $G'$ ;
4 Procedure DeriveCompact( $G, \rho, P$ )
5    $cnt \leftarrow 0; \Psi_{hx} \leftarrow$  all the instances of  $hx$ -pattern  $\psi_{hx}$  in  $G$ ;
6    $V_{\mathcal{F}} \leftarrow \{s\} \cup V \cup \Psi_{hx} \cup P \cup \{t\};$ 
7   foreach  $\psi_{hx} \in \Psi_{hx}$  do
8     foreach  $v \in \psi_{hx}$  do
9       add an edge  $\psi_{hx} \rightarrow v$  with capacity  $h - 1$ ;
10      add an edge  $v \rightarrow \psi_{hx}$  with capacity 1;
11   foreach  $\psi_{hx} \in P$  do
12      $cnt \leftarrow cnt$  of  $\psi_{hx}$ ;
13     foreach  $v \in \psi_{hx}$  and  $v \in G$  do
14       add an edge  $\psi_{hx} \rightarrow v$  with capacity  $h - 1$ ;
15       add an edge  $v \rightarrow \psi_{hx}$  with capacity  $1 + \frac{h-cnt}{cnt}$ ;
16        $deg_G(v, \psi_{hx}) \leftarrow deg_G(v, \psi_{hx}) + 1 + \frac{h-cnt}{cnt}$ ;
17   foreach  $v \in V$  do
18     add an edge  $v \rightarrow t$  with capacity  $\rho * h$ ;
19     add an edge  $s \rightarrow v$  with capacity  $deg_G(v, \psi_{hx})$ ;
20   Compute the minimum  $s - t$  cut  $(\mathcal{S}, \mathcal{T})$  from the flow network
     $\mathcal{F}(V_{\mathcal{F}}, E_{\mathcal{F}})$ ;
21   return  $G[S \setminus s]$ ;
```

[27] [28]. We prove that $G[S_1]$ and $G[S_2]$ are the same. First, we prove that $G[S_2]$ is a subgraph of $G[S_1]$ by contradiction. Suppose a connected component $G[S']$ of $G[S_2]$ is not hx -pattern ρ -compact, then there exists a subset $S' \subseteq S$ such that removing S' from S will result in removing less hx -patterns than $\rho \times |S'|$, then $|\psi_{hx}(G[S'])| - |\psi_{hx}(G[S \setminus S'])| < \rho \times |S'| = \rho \times (|S| - |S \setminus S'|)$. We have $|\psi_{hx}(G[S'])| - \rho \times |S'| < |\psi_{hx}(G[S \setminus S'])| - \rho \times |S \setminus S'|$. Therefore, replacing $G[S']$ by its subgraph $G[S \setminus S']$ in $G[S_2]$ will enlarge the value of $|\psi_{hx}(G[S_2])| - \rho \times |S_2|$, which contradicts the condition that $G[S_2]$ has the maximum $|\psi_{hx}(G[S_2])| - \rho \times |S_2|$. Second, we prove that $G[S_1]$ is a subgraph of $G[S_2]$ by contradiction. Suppose $G[S_1]$ is not a subgraph of $G[S_2]$, according to the result before, we have $S_2 \subset S_1$. There exists a subset $S \neq \emptyset$ and $S = S_1 \setminus S_2$. Removing S from $G[S_1]$ will result in removing at least $\rho \times |S|$ hx -patterns, then $|\psi_{hx}(G[S_1])| - |\psi_{hx}(G[S_2])| \geq \rho \times |S| = \rho \times (|S_1| - |S_2|)$. We have $|\psi_{hx}(G[S_1])| - \rho \times |S_1| \geq |\psi_{hx}(G[S_2])| - \rho \times |S_2|$, so enlarging $G[S_2]$ to $G[S_1]$ will not decrease the value of $|\psi_{hx}(G[S_2])| - \rho \times |S_2|$, which contradicts the condition that $G[S_2]$ has the maximum $|\psi_{hx}(G[S_2])| - \rho \times |S_2|$. Therefore, the theorem is proved. $\square$

According to Theorem 5, a basic verification algorithm is designed in Algorithm 4. We first derive all connected components of the hx -pattern $d_{\psi_{hx}}(G[S])$ -compact subgraph G' in G by DeriveCompact (Line 2). If $G[S]$ is a connected component of G' , the algorithm returns True (Line 3). In DeriveCompact, all the instances of hx -pattern is collected (Line 5). To build a flow network $\mathcal{F}(V_{\mathcal{F}}, E_{\mathcal{F}})$, a vertex set $V_{\mathcal{F}}$ is created, and vertices in $V_{\mathcal{F}}$ are linked by directed edges with different capacities (Lines 6-19). Then, the minimum $s - t$ cut $(\mathcal{S}, \mathcal{T})$ is computed (Line 20).

2) *Fast Verification Algorithm:* Although the basic verification algorithm can successfully verify whether a given subset is

L hx PDS, the scale of the flow network in Algorithm 4 is large, and the running time is long in large-scale graphs. We prove that the verification can be done by verifying the locally densest property only on the subgraph $G[S]$ and the vertices around the subgraph $G[S]$, which is denoted by $G[T]$. Since the size of $G[T]$ is much smaller than that of G , checking the minimum cut in $G[T]$ is much more efficient. Considering the complexity of the overlap of patterns, we propose a fast verification algorithm by constructing a smaller flow network based on $G[T]$. Based on the fact that only the hx -patterns at the boundary of $G[T]$ affect hx -pattern compact numbers in G , we use a set P to record these hx -patterns. For each $\psi_{hx}^{P_r} \in P$, the number of vertices that are contained in both $\psi_{hx}^{P_r}$ and $G[T]$ is cnt_{P_r} . The flow network $\mathcal{F}(V_{\mathcal{F}}, E_{\mathcal{F}})$ is shown in Fig. 7. The vertex set of $\mathcal{F}$ is $\{s\} \cup V \cup \Psi_{hx} \cup P \cup \{t\}$. We add the boundary hx -pattern set P into $V_{\mathcal{F}}$ to ensure that the results of solving the flow network of $G[T]$ are precisely consistent with that of G . The arc set of $\mathcal{F}$ is given as follows. The arcs for vertices and hx -patterns in $G[T]$ are the same as the former flow network. For each $\psi_{hx}^{P_r} \in P$, we add cnt_{P_r} incoming arcs of capacity $1 + \frac{h-cnt_{P_r}}{cnt_{P_r}}$ from the vertices that both in $\psi_{hx}^{P_r}$ and $G[T]$, and cnt_{P_r} outgoing arcs of capacity of $h - 1$ to the same set of vertices.

We then design a fast verification algorithm in Algorithm 5. The hx -pattern density of $G[S]$ is assigned to ρ (Line 1). Then, a breadth-first search is performed. U is used to store the vertices to be traversed (Line 4). The first vertex v from U is popped out (Line 6), and all hx -patterns containing v are iterated (Lines 7-25). For each ψ_{hx} that is not in W , if any vertex w in ψ_{hx} satisfying $\bar{\phi}_{hx}(w) < \rho$, ψ_{hx} will not affect the hx -pattern compact number of w (Lines 9-13); if any vertex w is in any outputted L hx PDS, False is assigned to VerifyL hx PDS (Lines 17-18); the number of vertices in ψ_{hx} satisfying $\phi_{hx}(w) \leq \rho$ is recorded and ψ_{hx} is added into P (Lines 19-25). All neighbors of v are iterated (Lines 26-30). For each neighbor w that is not in T , if $\phi_{hx}(w) > \rho$, False is assigned to VerifyL hx PDS (Line 28). If $\phi_{hx}(w) \leq \rho < \bar{\phi}_{hx}(w)$, w will be added into U and T (Line 30). If VerifyL hx PDS is False, a subgraph $G[T]$ induced by T and peripheral hx -patterns in P are used to compute all hx -pattern ρ -compact subgraphs in $G[T]$ via min-cut (Line 32). Finally, True is returned if $G[S]$ is maximal hx -pattern ρ -compact; otherwise, the algorithm returns False (Line 33). The flow network here is much smaller. Finally, we prove the correctness of the verification algorithm.

Theorem 6: Given a graph G and a self-densest subgraph $G[S]$, $G[S]$ is an L hx PDS of G if and only if the fast L hx PDS verification algorithm returns True.

Proof: On the one hand, if $G[S]$ is an L hx PDS of G , $G[S]$ is still an L hx PDS in $G[T]$, because only the hx -patterns in P might increase the hx -pattern compact numbers in $G[T]$ compared to the hx -pattern compact numbers in G . Otherwise, there exists a vertex v with hx -patterns in P contained in the maximal hx -pattern $d_{\psi_{hx}}(G[S])$ -compact subgraph containing $G[S]$, and we can construct a larger hx -pattern $d_{\psi_{hx}}(G[S])$ -compact subgraph in G by adding vertices with $\phi_{hx}(w) > d_{\psi_{hx}}(G[S])$ connected to v , which contradicts that $G[S]$ is an L hx PDS. On

Algorithm 5: Fast L hx PDS Verification Algorithm:

Input: $G = (V, E)$, S , $\Psi_{hx}(G)$, $\bar{\phi}_{hx}$, $\underline{\phi}_{hx}$
Output: VerifyLhxPDS

```

1  $\rho \leftarrow d_{\psi_{hx}}(G[S])$ , VerifyLhxPDS  $\leftarrow$  True, Valid  $\leftarrow$  True;
2  $U \leftarrow$  an empty queue,  $P \leftarrow \emptyset$ ,  $T \leftarrow \emptyset$ ,  $W \leftarrow \emptyset$ ,  $cnt \leftarrow 0$ ;
3 foreach  $u \in S$  do
4   if  $u \notin T$  then push  $u$  to  $U$ , insert  $u$  into  $T$ ;
5   while  $U$  is not empty do
6      $v \leftarrow$  pop out the front vertex in  $U$ ;
7     foreach  $\psi_{hx} \in \Psi_{hx}(G)$  where  $v \in \psi_{hx}$  do
8       Valid  $\leftarrow$  True;
9       if  $\psi_{hx} \notin W$  then
10        foreach  $w \in \psi_{hx}$  do
11          if  $\bar{\phi}_{hx}(w) < \rho$  then Valid  $\leftarrow$  False;
12          insert  $\psi_{hx}$  into  $W$ ;
13        else Valid  $\leftarrow$  False;
14        if Valid then
15           $cnt \leftarrow 1$ ;
16          foreach  $w \in \psi_{hx}$  and  $w \neq v$  do
17            if  $w \notin T$  and  $w$  is in any LhxPDS then
18              VerifyLhxPDS  $\leftarrow$  False;
19            if  $\underline{\phi}_{hx}(w) \leq \rho$  then
20              if  $w \notin T$  then
21                push  $w$  to  $U$ , insert  $w$  into  $T$ ;
22               $cnt \leftarrow cnt + 1$ ;
23          if  $cnt \neq h$  and  $\psi_{hx} \notin P$  then
24            insert  $\psi_{hx}$  and  $cnt$  into  $P$ ;
25            VerifyLhxPDS  $\leftarrow$  False;
26        foreach  $(v, w) \in E$  do
27          if  $w \notin T$  and  $\underline{\phi}_{hx}(w) > \rho$  then
28            VerifyLhxPDS  $\leftarrow$  False;
29          else if  $w \notin T$  and  $\bar{\phi}_{hx}(w) > \rho$  then
30            push  $w$  to  $U$ , add  $w$  into  $T$ ;
31 if VerifyLhxPDS then return True;
32  $G' \leftarrow$  DeriveCompact( $G[T]$ ,  $\rho - \frac{1}{|V(G[T])|^2}$ ,  $P$ );
33 return  $G[S]$  is a connected component in  $G'$ ;

```

the other hand, if $G[S]$ is not an L hx PDS of G , we will find a larger hx -pattern $d_{\psi_{hx}}(G[S])$ -compact subgraph containing $G[S]$ in $G[T]$. Therefore, $G[S]$ is not an L hx PDS in $G[T]$, and the algorithm returns False. Therefore, the algorithm returns True only when $G[S]$ is an L hx PDS. $\square$

E. L hx PDS Discovery Algorithm in Signed Graphs (IPPV-S)

Combining all the algorithms above, we derive the locally antagonistic balanced pattern densest subgraph discovery algorithm in signed graphs, called the IPPV-S algorithm shown in Algorithm 6. An empty stack st is initialized, and G' is assigned to G (Line 1). The bounds of hx -pattern compact numbers are initialized via InitializeBd (Line 2). L hx PDS candidates are derived via ProposeCL and Prune (Line 4-5). Next, the L hx PDS candidates in S are reversely pushed into st , and the first L hx PDS candidate in st , the one with the highest ϕ_{hx} value, is popped out (Lines 6-7). The L hx PDS candidate is verified by IsDensest (Line 8) and VerifyL hx PDS (line 9). If $G[S]$ is an L hx PDS, it will be putted into S' , and k is decreased by 1 (Line 10). If $G[S]$ is not an L hx PDS but is

Algorithm 6: Iterative Propose-Prune-and-Verify Algorithm Based on Convex Programming (IPPV-S):

Input: $G = (V, E)$, number of iterations T , an integer k
Output: top- k LhxPDSes

```

1  $st \leftarrow$  an empty stack;  $G' \leftarrow G$ ;  $ant \leftarrow \emptyset$ ;
2  $\bar{\phi}_{hx}, \underline{\phi}_{hx} \leftarrow$  InitializeBd( $G', h$ );
3 while  $k > 0$  do
4    $S, \bar{\phi}_{hx}, \underline{\phi}_{hx} \leftarrow$  ProposeCL( $G', T, \bar{\phi}_{hx}, \underline{\phi}_{hx}$ );
5    $S \leftarrow$  Prune( $G', S, \bar{\phi}_{hx}, \underline{\phi}_{hx}$ );
6   foreach  $S \in S$  reversely do push  $S$  into  $st$ ;
7    $S \leftarrow$  pop out the top stable group from  $st$ ;
8   if IsDensest( $G[S]$ ) then
9     if VerifyLhxPDS( $G, S, \bar{\phi}_{hx}, \underline{\phi}_{hx}$ ) then
10      put  $S$  into  $S'$ ;  $k \leftarrow k - 1$ ;
11     if  $st$  is empty then break;
12      $S \leftarrow$  pop out the top stable group from  $st$ ;
13    $G' \leftarrow G[S]$ ;
14 foreach  $S \in S'$  do
15    $V_L, V_R \leftarrow$  PASTA-toss( $S'$ );
16    $pol \leftarrow \frac{2 \cdot (|E_L^+| + |E_R^+| - |E_L^-| - |E_R^-| + |E_{LR}^-| - |E_{LR}^+|)}{|V_L| + |V_R|}$ ;
17    $icoh \leftarrow \frac{|E(S)|}{\binom{|S|}{2}}$ ;
18    $eand \leftarrow \frac{|E_{LR}^-|}{(|V_L| + |V_R|)^2}$ ;
19    $ant(S) \leftarrow p \cdot icoh \cdot eand$ ;
20 sort  $S$  in  $S'$  in descending order according to  $ant$ ;
21 return  $S'$ ;

```

self-densest, S is updated as the top L hx PDS candidate from st (Line 12). Then, $G[S]$ is assigned to G' for the next iteration (Line 13). The above process is repeated until top- k L hx PDSes are found (line 3) or the stack is empty (Line 11). For each L hx PDS in signed graph, the left and right sets of the L hx PDS is computed by PASTA-toss algorithm [46] (Line 15). The polarity, internal cohesion, and external antagonistic normalized density are calculated (Lines 16-18). The antagonism of the L hx PDS is proposed (Line 19). The L hx PDSes are sorted and outputted based on ant values descendingly (Lines 20-21).

Complexity Analysis. We analyze the complexity of IPPV-S by distinguishing its stages. First, in the Propose stage, we use T to denote the number of iterations that SEQ-kClist++ needs. Each iteration costs $O(n + |\Psi_{hx}|)$. Second, in the Prune stage, we use N_{CL} to represent the total number of L hx PDS candidates ($N_{CL} \ll n$), where processing each candidate costs $O(n + |\Psi_{hx}|)$. Third, the Verify stage dominates the computational cost. N_{Flow} is the number of times IsDensest and VerifyL hx PDS are called. Crucially, the verification is performed on local subgraphs. Let n' and $|\Psi'_{hx}|$ denote the number of vertices and patterns in the local verification graph, where $n' \ll n$ and $|\Psi'_{hx}| \ll |\Psi_{hx}|$. The time complexity of max-flow computation using Dinic's Algorithm is $O((n' + |\Psi'_{hx}|)^2 \cdot (n' + |\Psi'_{hx}| \cdot h))$. Additionally, the time complexity of the post-processing PASTA-toss is $O(n^2 + nm)$. Consequently, the total time complexity of IPPV-S is $O((T + N_{CL}) \cdot (n + |\Psi_{hx}|) + N_{Flow} \cdot (n' + |\Psi'_{hx}|)^2 \cdot (n' + |\Psi'_{hx}| \cdot h) + (n^2 + nm) \cdot k)$. As $(T + N_{CL}) \cdot (n + |\Psi_{hx}|) < (n^2 + nm) \cdot k \ll N_{Flow} \cdot (n' + |\Psi'_{hx}|)^2 \cdot (n' + |\Psi'_{hx}| \cdot h)$, the time complexity is dominated by $O(N_{Flow} \cdot (n' + |\Psi'_{hx}|)^2 \cdot (n' + |\Psi'_{hx}| \cdot h))$. Since the

TABLE IV
EFFICIENCY OF IPPV-S AND Bansal

Dataset	IPPV-s (hx=3b)	IPPV-s (hx=4f)	Bansal
BA	0.17(s)	0.18	1.00
RH	17.25	20.25	19.00
SS	112.03	92.28	267.00
WC	92406.82	174221.11	4207.00
DBLP-20p	364.19	450.48	4235.00

verification is strictly restricted to small local structures, the algorithm achieves high scalability on sparse, large-scale networks. The space complexity is linear, $O(n + |\Psi_{hx}|)$, for storing graph and pattern structures.

VI. EXPERIMENTS

A. Experimental Setup

In this section, we present our experimental results. The datasets we use are signed real-world graphs [47], [48] spanning trust, voting, and conflict domains, which are primarily used to evaluate the effectiveness of the proposed model. All datasets are listed in Table II. Besides, we use large-scale real unsigned social, collaboration, and product networks [47], [48] (e.g., Amazon, DBLP), which are utilized to assess the algorithmic efficiency and scalability on massive datasets and listed in Table III. We randomly assign the edge signs in a certain proportion and observe the efficiency performance.

We compare the performances of our algorithm to the following algorithms:

- **Balanced structure discovery:** **1) BCE:** an maximal balanced clique enumeration algorithm proposed by Yao et al. [16]. **2) BANSAL:** a 3-approximation algorithm for 2-Correlation-Clustering on signed graphs proposed by Bansal et al. [49].
- **Polarized community search:** **3) EIGENSIGN:** an efficient algorithm for 2-Polarized-Communities with provable approximation guarantees proposed by Bonchi et al. [18]. **4) ELEC-TRON:** a heuristic to find polarized communities proposed by Niu et al. [19].

Note: Since unsigned density methods fail to capture antagonism, they serve as theoretical references in the Case Study rather than primary baselines. All algorithms are implemented in C++ and compiled by g++ compiler at -O3 optimization level. All experiments are evaluated on a machine with Intel(R) Xeon(R) CPU 3.20 GHz processor and 256 GB memory, with Ubuntu operating system. Algorithms running for more than 48 hours are forcibly terminated.

B. Efficiency

1) Influence of the Proportion of Negative Edges: By observing the signed graph statistics, we find that the proportion of negative edges in a graph significantly affects the number of the antagonistic balanced patterns, and consequently influences the efficiency of the algorithm. The proportions of negative edges in real-world signed graphs are constant, to explore the impact of negative edge ratios on efficiency, we use five large unsigned

graph datasets. The statistics of the unsigned graph datasets are listed in Table III.

The number of antagonistic balanced patterns and the proportion of negative edges show a correlation across all datasets. When the proportion of negative edges is low, the number of antagonistic balanced patterns increases as the proportion increases; there is a maximum number of antagonistic balanced patterns when the proportion of negative edges is around 60%, which is related to the edge proportion of antagonistic balanced pattern itself; when the proportion of negative edges is high, the number of antagonistic balanced patterns decreases due to the increase in the number of all-negative-edge patterns. The experiments in Fig. 8 indicate a direct relationship, where an increase of the number of antagonistic balanced patterns corresponds to a proportional increase in execution time. This trend is consistently observed across different datasets. The only deviation is observed in the youtube and lastfm datasets when $hx = 4f$, where the running time decreases when the number of antagonistic balanced patterns increases. In addition to the total number of patterns, the filtering and calculation of antagonistic balanced patterns also affect the efficiency of the algorithm.

2) Efficiency vs Existing Algorithm: We compare IPPV-s with the algorithm Bansal by setting $hx = 3b$ and $hx = 4f$ respectively, and the results are shown in Table IV. We compare IPPV-s with Bansal on five datasets, and IPPV-s is more efficient on most of the datasets. The only exception is on the wikiconflict dataset, where we believe is directly related to the high proportion of negative edges in this dataset. The running time of our algorithm is closely related to the size of the graph and the number of antagonistic balanced patterns. A rise in the number of antagonistic balanced patterns can cause an increase in running time. The existing heuristic algorithms are efficient but lack theoretical guarantees, and no comparisons are made in this paper.

3) Influence of Structural Property: We discuss how the density property ($|E|/|V|$) of graph datasets influences the efficiency of IPPV-s. As shown in Fig. 9, we set $hx = 3b$ and $4f$, and select four datasets and randomly sample edges of these datasets in different proportions. According to the outputs from these synthetically generated graphs, the running time increases proportionally as the density of datasets increases. The reason is that when density is higher, the number of hx -patterns increases, lengthening the total running time.

C. Ablation Study

We evaluate component contributions by comparing IPPV-s against three variants: w/o DeriveSG (using loose bounds), w/o Prune (disabling Algorithm 3), and Basic-verify (using baseline verification algorithm).

1) Effectiveness of Tightening and Pruning: We investigate the impact of the tightening and pruning modules on the candidate generation phase. As shown in Table V, the pruning strategy is fundamental for tractability; without it, the algorithm fails to terminate due to an exponential explosion of ineligible candidates entering the verification phase. The stable group derivation strategy effectively refines the search space; removing

TABLE V
ABLATION STUDY ON REPRESENTATIVE DATASETS

Dataset	Variant	Candidate Nodes	Running Time (s)
RH	IPPV-s (hx=3b)	144	17.3
	w/o DeriveSG	148	18.6
	w/o Prune	N/A	>24h
WC	IPPV-s (hx=3b)	335	92406.8
	w/o DeriveSG	345	92567.3
	w/o Prune	N/A	>24h

TABLE VI
CHARACTERISTICS OF THE L_{hx} PDSes WITH VARYING h AND x

Pattern	$ V_L $	$ V_R $	ANT	POL	ICOH	EAND
3b	20	2	2.725	15.546	0.922	0.190
4a	34	0	0.000	31.941	1.000	0.000
4b	34	0	0.000	31.941	1.000	0.000
4c	58	7	0.358	22.831	0.386	0.041
4d	20	2	2.725	15.546	0.922	0.190
4e	20	2	2.725	15.546	0.922	0.190
4f	20	1	2.062	15.524	0.976	0.136

it results in looser bounds and generates more candidate nodes for verification. Although the total time saving is sometimes marginal due to the overhead of tightening itself, this module guarantees the compactness and exactness of the candidates.

2) *Effectiveness of Fast Verification*: We evaluate the acceleration provided by our optimized verification algorithm. We use L_{hx} PDS to represent IPPV-s algorithm of hx -pattern. T_{basic} and T_{fast} represent the running time of IPPV-s algorithm with Algorithms 4 and 5, respectively. Fig. 10 shows the running time difference ΔT between basic and fast verification algorithms, $\Delta T = T_{basic} - T_{fast}$. The fast verification method with a smaller flow network is faster than basic verification algorithm. Especially as dataset size increases, the efficiency gap between the two algorithms becomes more apparent. The results demonstrate the importance and benefit of optimizing the verification algorithm.

D. Characteristics of the Detected L_{hx} PDSes

We adopt the proposed quality measures to show the characteristics of the detected top-1 L_{hx} PDSes. These measures include antagonism (ANT), polarity (POL), internal cohesion (ICOH), and external antagonistic normalized density (EAND). Then we compare our detected L_{hx} PDSes with those detected by existing algorithms.

1) *Characteristics With Varying h and x* : We use a network of the correlates of war data [10] from 1996 to 1999 to analyze the characteristics of detected L_{hx} PDSes with varying h and x . There are 151 vertices, 1100 positive edges, and 147 negative edges in the network. The vertices represent different countries; the positive edges reflect cooperative ties through formal alliance memberships, supranational unions, or binding intergovernmental agreements; the negative edges indicate adversarial relations via active warfare, non-military diplomatic conflicts, territorial disputes, or ideological and policy opposition. The characteristics of the top-1 L_{hx} PDSes with varying h and x is shown in Table VI. The results of pattern 3b, 4d, 4e are the same subgraph with the highest antagonism. As shown in Fig. 2, pattern 4a and 4b do not include any unbalanced pattern.

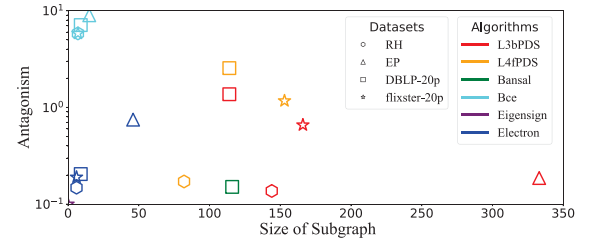


Fig. 12. Antagonism and size of the results of different algorithms.

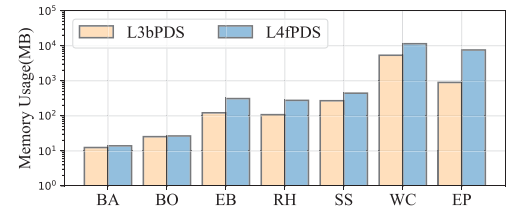


Fig. 13. Memory usage of our algorithms.

which turns the solution into finding dense structures similar to unsigned graphs. To verify this, we performed an unsigned densest subgraph detection on the same dataset. It yielded the exact same set of 34 vertices as patterns 4a and 4b, a clique with high cohesion and polarity yet zero antagonism. This isolates the marginal contribution of structural balance, proving that simple patterns are insufficient for mining signed networks, as they are dominated by topological density and fail to capture the specific structural antagonism. The result of pattern 4c has the largest size, but has a relatively low internal cohesion. The result of pattern 4f is slightly worse than the optimal result as one vertex is missing.

We visualize the subgraph with the highest antagonism in Fig. 11. In this visualization, blue solid lines denote alliances and red dashed lines denote conflicts. The subgraph, featuring a left set of 20 North American and European nations (BEL, USA,...,HUN, POL,...) and a right set of Russia (RUS) and Iraq (IRQ), encapsulates critical dynamics of the post-Cold War unipolar order and emerging challenges. This structure reflects two distinct geopolitical narratives: Russia's contested reintegration into Euro-Atlantic institutions and Iraq's isolation under U.S.-led sanctions, offering insights into power transitions, alliance fragmentation, and institutional constraints during this pivotal era. It also anticipates future crises: Russia's resentment over NATO expansion foreshadows regional conflicts in 2014 and 2022, while Iraq's destabilization under sanctions laid the groundwork for the 2003 Iraq War.

We also compare the results with existing algorithms. The result of EElectron is a graph ($|V_L| = 5$, $|V_R| = 6$) with negative antagonism as the polarity ($POL = -0.182$) is negative. The result of Bce is a 4-clique ($|V_L| = 2$, $|V_R| = 2$), which is a quite small size. Bansal and Eigensign algorithm failed to find valid subgraphs. The results of all existing algorithms cannot reflect the international situation. We believe that the subgraphs we mined have practical value and significance for the analysis of international relations, especially in the focus on conflicts and confrontations. In the next section, we compare the characteristics of our detected L_{hx} PDSes with the results of existing algorithms on more datasets.

TABLE VII
CHARACTERISTICS OF THE RESULTS OF DIFFERENT ALGORITHMS

Algorithm	$ V_L $	$ V_R $	ANT	POL	ICOH	EAND
L3bPDS	136	199	25.377	89.385	0.481	0.590
L4fPDS	129	186	31.940	102.165	0.510	0.613
Bansal	254	250	-0.001	-0.012	0.297	0.164
Bce	3	3	5.000	5.000	1.000	1.000
Eigensign	1218	1225	0.002	0.862	0.080	0.033
Electron	38	39	0.121	0.442	0.603	0.453

2) *Characteristics V.s. Existing Algorithms:* We compare the quality measures of our algorithms with existing algorithms. We analyze the measures of all results in detail on the wikiconflict dataset and compare the size and antagonism of the results of different algorithms on multiple datasets.

The characteristics of the results of different algorithms on wikiconflict dataset are shown in Table VII. The Eigensign algorithm find the largest subgraph among all algorithms, but the properties of the subgraph are not good, which is a subgraph that is not dense and not antagonistic. The Bce algorithm can find the subgraph with the highest internal cohesion and external antagonistic normalized density, but the size is too small to obtain a subgraph with high antagonism, which is also the limitation of finding strictly balanced cliques in a graph. The results of Bansal and Electron algorithms have no obvious features. The results of the IPPV-s algorithms when $hx = 3b$ and $hx = 4f$ have the second-highest and highest antagonism, which shows that our algorithms are high-quality solutions for antagonistic dense subgraph problem.

We also compare the size and antagonism of the subgraphs of different algorithms on multiple datasets. The size and antagonism of the results of all algorithms on four datasets is shown in Fig. 12. For the algorithms that output multiple subgraphs, we select the subgraph with the highest antagonism for comparison. Subgraphs with negative antagonism are filtered out, therefore, several results of Bansal and Eigensign algorithms are not displayed in the figure. The Bce algorithm find the subgraphs with higher antagonism, but the size is too small to extract more useful information for subsequent analysis. The results of Electron algorithm also have no obvious features. The results of the IPPV-s algorithms when $hx = 3b$ and $hx = 4f$ have relative high antagonism and considerable size, which is in line with the goal of finding antagonistic near-balanced dense subgraph. Even in datasets where the proportion of negative edges is relatively low, useful subgraph results can still be found.

E. Memory Overheads

We compare the memory utilization for the IPPV-s algorithms when $hx = 3b$ and $hx = 4f$ across all datasets ($k = 5$). Fig. 13 illustrates a clear correlation between memory usage and dataset size. An increase in the number of dataset-related patterns is accompanied by a corresponding increase in memory usage. IPPV-s strategically reduces the size of candidate subgraphs through a pruning mechanism prior to evaluating self-compactness. The verifying part often dominates the memory consumption.

VII. CONCLUSION

In this paper, we address the discovery of antagonistic near-balanced dense subgraphs by introducing the unified Antagonism measure. We propose IPPV-s, an efficient “propose-prune-and-verify” pipeline that leverages convex programming bounds and a tailored fast verification algorithm to identify top- k L hx PDS. Extensive experiments on real datasets show the high efficiency and scalability of our proposed algorithm. Despite the effectiveness of IPPV-s, the computational cost of maximum flow verification remains a constraint for billion-scale or extremely dense networks, and the reliance on static, pre-defined structures limits the detection of evolving conflicts. Future work will enhance scalability by developing approximation algorithms and leveraging parallel computing paradigms. We also aim to extend the framework to dynamic signed networks to capture the temporal evolution of conflicts, facilitating a deeper understanding of adversarial behaviors in complex real-world scenarios.

REFERENCES

- [1] X. Song, Y. Chi, K. Hino, and B. L. Tseng, “Identifying opinion leaders in the blogosphere,” in *Proc. Int. Conf. Inf. Knowl. Manage.*, 2007, pp. 971–974.
- [2] J. Leskovec, D. P. Huttenlocher, and J. M. Kleinberg, “Signed networks in social media,” in *Proc. SIGCHI Conf. Hum. Factors Comput. Syst.*, 2010, pp. 1361–1370.
- [3] A. Vinayagam et al., “Integrating protein-protein interaction networks with phenotypes reveals signs of interactions,” *Nature Methods*, vol. 11, no. 1, pp. 94–99, 2014.
- [4] J. Tang, C. C. Aggarwal, and H. Liu, “Recommendations in signed social networks,” in *Proc. 25th Int. Conf. World Wide Web*, 2016, pp. 31–40.
- [5] F. Harary, “On the notion of balance of a signed graph,” *Michigan Math. J.*, vol. 2, pp. 143–146, 1953.
- [6] D. Cartwright and F. Harary, “Structural balance: A generalization of Heider’s theory,” *Psychol. Rev.*, vol. 63, no. 5, pp. 277–293, 1956.
- [7] R. Figueiredo and Y. Frota, “The maximum balanced subgraph of a signed graph: Applications and solution approaches,” *Eur. J. Oper. Res.*, vol. 236, pp. 473–487, 2014.
- [8] Z. Chen, L. Yuan, X. Lin, L. Qin, and W. Zhang, “Balanced clique computation in signed networks: Concepts and algorithms,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 11, pp. 11079–11092, Nov. 2023.
- [9] M. Isoraite, “Importance of strategic alliances in company’s activity,” *Intellectual Econ.*, vol. 1, pp. 39–46, 2009.
- [10] P. Doreian and A. Mrvar, “Structural balance and signed international relations,” *J. Social Struct.*, vol. 16, pp. 1–49, 2015.
- [11] S. Kumar, W. L. Hamilton, J. Leskovec, and D. Jurafsky, “Community interaction and conflict on the web,” in *Proc. World Wide Web Conf.*, 2018, pp. 933–943.
- [12] G. A. Miller, “WordNet: A lexical database for english,” *Commun. ACM*, vol. 38, pp. 39–41, 1995.
- [13] V. Kumar, N. Joshi, A. Mukherjee, G. Ramakrishnan, and P. Jyothi, “Cross-lingual training for automatic question generation,” in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 4863–4872.
- [14] A. Krishnan, P. Deepak, S. Ranu, and S. Mehta, “Leveraging semantic resources in diversified query expansion,” *World Wide Web*, vol. 21, pp. 1041–1067, 2017.
- [15] B. Ordozgoiti, A. Matakos, and A. Gionis, “Finding large balanced subgraphs in signed networks,” in *Proc. Web Conf.*, 2020, pp. 1378–1388.
- [16] K. Yao, L. Chang, and L. Qin, “Identifying large structural balanced cliques in signed graphs,” *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 3, pp. 1145–1160, Mar. 2024.
- [17] K. Zhang, D. Lo, E.-P. Lim, and P. K. Prasetyo, “Mining indirect antagonistic communities from social interactions,” *Knowl. Inf. Syst.*, vol. 35, pp. 553–583, 2012.
- [18] F. Bonchi, E. Galimberti, A. Gionis, B. Ordozgoiti, and G. Ruffo, “Discovering polarized communities in signed networks,” in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 961–970.

- [19] J. Niu and A. E. Saryüce, "On cohesively polarized communities in signed networks," in *Proc. Companion Proc. ACM Web Conf.*, 2023, pp. 1339–1347.
- [20] F. Hao, S. S. Yau, G. Min, and L. T. Yang, "Detecting k-balanced trusted cliques in signed social networks," *IEEE Internet Comput.*, vol. 18, pp. 24–31, 2014.
- [21] R. Li et al., "Efficient signed clique search in signed networks," in *Proc. IEEE 34th Int. Conf. Data Eng.*, 2018, pp. 245–256.
- [22] L. Chu, Z. Wang, J. Pei, J. Wang, Z. Zhao, and E. Chen, "Finding gangs in war from signed networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1505–1514.
- [23] F. Yang, H. Ma, C. Yan, Z. Li, and L. Chang, "Polarized communities search via co-guided random walk in attributed signed networks," *ACM Trans. Internet Technol.*, vol. 23, pp. 1–22, 2023.
- [24] Z. Zhang et al., "Signed graph representation learning: A survey," 2024, *arXiv:2402.15980*.
- [25] Z. Yang, Y. Li, B. Yu, J. Hu, and H. Chen, "A degree-corrected stochastic block model for community discovery in signed networks with heterogeneous degree distributions," *Pattern Recognit.*, vol. 171, Part B, 2026, Art. no. 112285.
- [26] J.-C. Picard and M. Queyranne, "A network flow solution to some non-linear 0-1 programming problems, with applications to graph theory," *Networks*, vol. 12, pp. 141–159, 1982.
- [27] A. V. Goldberg, "Finding a maximum density subgraph," Univ. California, Berkeley, CA, USA, 1984.
- [28] Y. Fang, K. Yu, R. Cheng, L. V. S. Lakshmanan, and X. Lin, "Efficient algorithms for densest subgraph discovery," *Proc. VLDB Endow.*, vol. 12, pp. 1719–1732, 2019.
- [29] M. Charikar, "Greedy approximation algorithms for finding dense components in a graph," in *Proc. Int. Workshop Approximation Algorithms Combinatorial Optim.*, 2000, pp. 84–95.
- [30] M. Danisch, T.-H. H. Chan, and M. Sozio, "Large scale density-friendly graph decomposition via convex programming," in *Proc. 26th Int. Conf. World Wide Web*, 2017, pp. 233–242.
- [31] D. Boob et al., "Flowless: Extracting densest subgraphs without flow computations," in *Proc. Web Conf.*, 2019, pp. 573–583.
- [32] C. Chekuri, K. Quanrud, and M. R. Torres, "Densest subgraph: Supermodularity, iterative peeling, and flow," in *Proc. ACM-SIAM Symp. Discrete Algorithms*, 2022, pp. 1531–1555.
- [33] C. Tsourakakis, "The K-clique densest subgraph problem," in *Proc. 24th Int. Conf. World Wide Web*, 2015, pp. 1122–1132.
- [34] M. Mitzenmacher, J. W. Pachocki, R. Peng, C. E. Tsourakakis, and S. C. Xu, "Scalable large near-clique detection in large-scale networks via sampling," in *Proc. 21th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2015, pp. 815–824.
- [35] B. Sun, M. Danisch, T. H. Chan, and M. Sozio, "KClust++: A simple algorithm for finding K-clique densest subgraphs in large graphs," *Proc. VLDB Endowment*, vol. 13, pp. 1628–1640, 2020.
- [36] L. Qin, R.-H. Li, L. Chang, and C. Zhang, "Locally densest subgraph discovery," in *Proc. 21th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, pp. 965–974, 2015.
- [37] T. B. Trung, L. Chang, N. T. Long, K. Yao, and H. T. T. Binh, "Verification-free approaches to efficient locally densest subgraph discovery," in *Proc. IEEE 39th Int. Conf. Data Eng.*, 2023, pp. 1–3.
- [38] C. Ma, R. Cheng, L. V. Lakshmanan, and X. Han, "Finding locally densest subgraphs: A convex programming approach," *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 2719–2732, 2022.
- [39] R. Samusevich, M. Danisch, and M. Sozio, "Local triangle-densest subgraphs," in *Proc. IEEE/ACM Int. Conf. Adv. Social Netw. Anal. Mining*, 2016, pp. 33–40.
- [40] X. Xu, H. Liu, X. Lv, Y. Wang, and D. Li, "An efficient and exact algorithm for locally h-clique densest subgraph discovery," *Proc. ACM Manage. Data*, vol. 2, no. 6, pp. 1–26, 2024.
- [41] S. Wuchty, Z. N. Oltvai, and A. L. Barabasi, "Evolutionary conservation of motif constituents in the yeast protein interaction network," *Nature Genet.*, vol. 35, pp. 176–179, 2003.
- [42] J. Leskovec, A. Singh, and J. M. Kleinberg, "Patterns of influence in a recommendation network," in *Proc. Pacific-Asia Conf. Knowl. Discov. Data Mining*, 2006, pp. 380–389.
- [43] J. Hu et al., "Discovering maximal motif cliques in large heterogeneous information networks," in *Proc. IEEE 35th Int. Conf. Data Eng.*, 2019, pp. 746–757.
- [44] D. Goldfarb and S. Liu, "An $O(n^3)$ primal interior point algorithm for convex quadratic programming," *Math. Programm.*, vol. 49, pp. 325–340, 1991.
- [45] S. P. Boyd and L. Vandenberghe, "Convex optimization," *IEEE Trans. Autom. Control*, vol. 51, pp. 1859–1859, 2010.

- [46] T. Coleman, J. Saunderson, and A. Wirth, "A local-search 2-approximation for 2-correlation-clustering," in *Embedded Systems and Applications*. Berlin, Germany: Springer, 2008.
- [47] J. Leskovec and A. Krevl, "SNAP datasets: Stanford large network dataset collection," Jun. 2014. [Online]. Available: <http://snap.stanford.edu/data>
- [48] R. A. Rossi and N. K. Ahmed, "The network data repository with interactive graph analytics and visualization," in *Proc. AAAI Conf. Artif. Intell.*, pp. 4292–4293, 2015.
- [49] N. Bansal, A. Blum, and S. Chawla, "Correlation clustering," *Mach. Learn.*, vol. 56, pp. 89–113, 2004.



Xiaojia Xu received the BS and PhD degrees from the School of Information, Renmin University of China, in 2020 and 2026, respectively. She is currently an assistant professor with the Department of Software Engineering, Beijing University of Technology. Her research interests include algorithm design and analysis, wireless networks, and operations research.



Haoyu Liu received the BS degree from the School of Computer Science and Technology, Xidian University, Xi'an, China, and the MS degree from the School of Information, Renmin University of China, Beijing, China. His research interests include algorithm design and analysis, graph theory, and social networks.



Xiaowei Lv received the BS degree from the Faculty of Information Science and technology, Ocean University of China, in 2023. He is currently working toward the MS degree with the School of Information, Renmin University of China. His research interests include algorithm design and analysis, large language models, and generative recommendation.



Yongcai Wang received the BS and PhD degrees from the Department of Automation Sciences and Engineering, Tsinghua University, in 2001 and 2006, respectively. He worked as associated researcher with NEC Labs. China from 2007–2009. He was a research scientist with Institute for Interdisciplinary Information Sciences (IIIS), Tsinghua University from 2009–2015. He was a visiting scholar with Cornell University in 2015. He is currently professor with the Department of Computer Sciences, Renmin University of China. His research interests include intelligent network, combinatorial optimization algorithms, and applications.



Deying Li received the MS degree in mathematics from Huazhong Normal University, in 1988, and the PhD degree in computer science from the City University of Hong Kong, in 2004. She is currently a professor with the Department of Computer Science, Renmin University of China. Her research includes wireless networks, mobile computing and algorithm design, and analysis.